
Курс: Рекомендательные системы

Занятие 19:

Рекомендательный сервис

Лашинин Олег

03.06.26

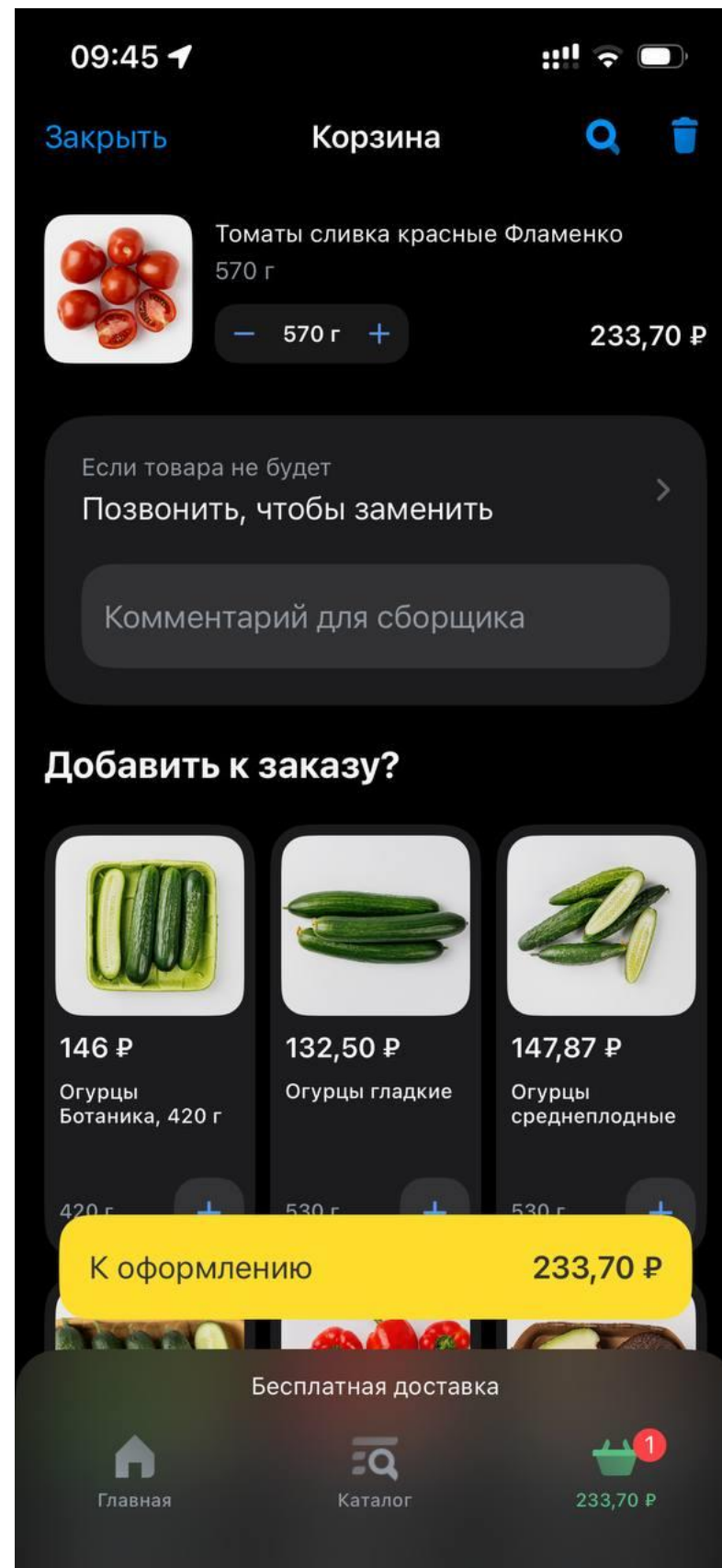
Почему сегодня эта тема?

Все, что мы разобрали, нужно как-то доставить до
пользователей

План занятия

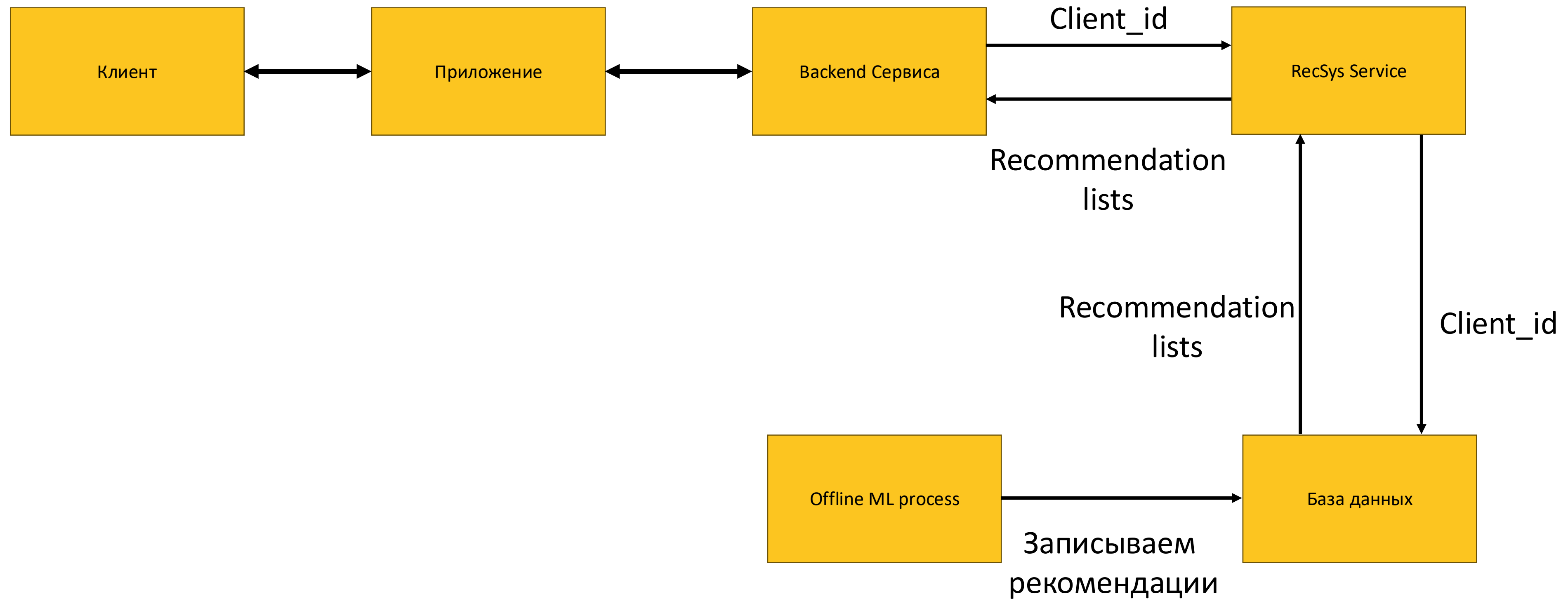
- Простой сервис с готовыми рекомендациями
- Простой сервис с пересчетом рекомендаций по контексту
- Пагинация
- Сервис с near real time inference
- Сервис с онлайн инференсом

Рассмотрим на примере сервиса этого сервиса



- 1) Нужно сделать рекомендации, дополняющие корзину
- 2) Есть история пользователя из прошлого, есть контекст текущей корзины
- 3) Есть разные магазины
- 4) Есть необходимость в пагинации
- 5) Есть проблемы холодного старта
- 6) Есть проблема диверсификации
- 7) Есть экономическая составляющая
- 8) Есть потенциальная реклама (продвижение)

Рассмотрим на примере сервиса этого сервиса



Хранение списков рекомендаций

User_id	recommendations	Datetime_of_update
1	[1, 2, 3, 4, 5]	1345
2	[1, 12, 3, 4, 5]	1234
3	[1, 42, 12, 43]	2435

Плюсы:

- 1) Легко сделать
- 2) Быстрое время ответа

Минусы:

- 1) Может быть сложно одновременно обновлять и держать нагрузку на ответы
- 2) Долгое время обновления рекомендаций
- 3) Пустая трата места (не каждый юзер зайдет)
- 4) Список фиксированной длины, не сохраняются рекомендации дальше

Хранение списков рекомендаций: оптимизация

User_id	recommendations	Datetime_of_update
1	[1, 2, 3, 4, 5]	1345
2	[1, 12, 3, 4, 5]	1234
3	[1, 42, 12, 43]	2435

Важен размер БД, если это критично, то методы оптимизации:

- 1) Сократить место на хранение 1 элемента списка (item_id строки -> маппинг в item_id Uint)
- 2) Сократить число юзеров, по остальным выдавать дефолт
- 3) Сократить размер списков рекомендаций для всех
- 4) Гибридно посчитать списки длины рекомендаций, для кого-то побольше, для кого-то поменьше.
- 5) Делать рекомендации полуперсонально

Хранение списков рекомендаций: оптимизация

User_id	recommendations	Datetime_of_update
1	[1, 2, 3, 4, 5]	1345
2	[1, 12, 3, 4, 5]	1234
3	[1, 42, 12, 43]	2435

Как подобрать размер длины списков?

- 1) Можно посмотреть распределение на длину скрола, и покрыть X% всех просмотров (дальше можно показать дефолтные подборки)
- 2) Можно провести тесты на деградацию/перегрузку БД в зависимости от длины списков и найти оптимальные точки
- 3) Можно неактивным юзерам делать короткие списки, активным юзерам списки подлиннее показывать

Хранение списков рекомендаций: оптимизация

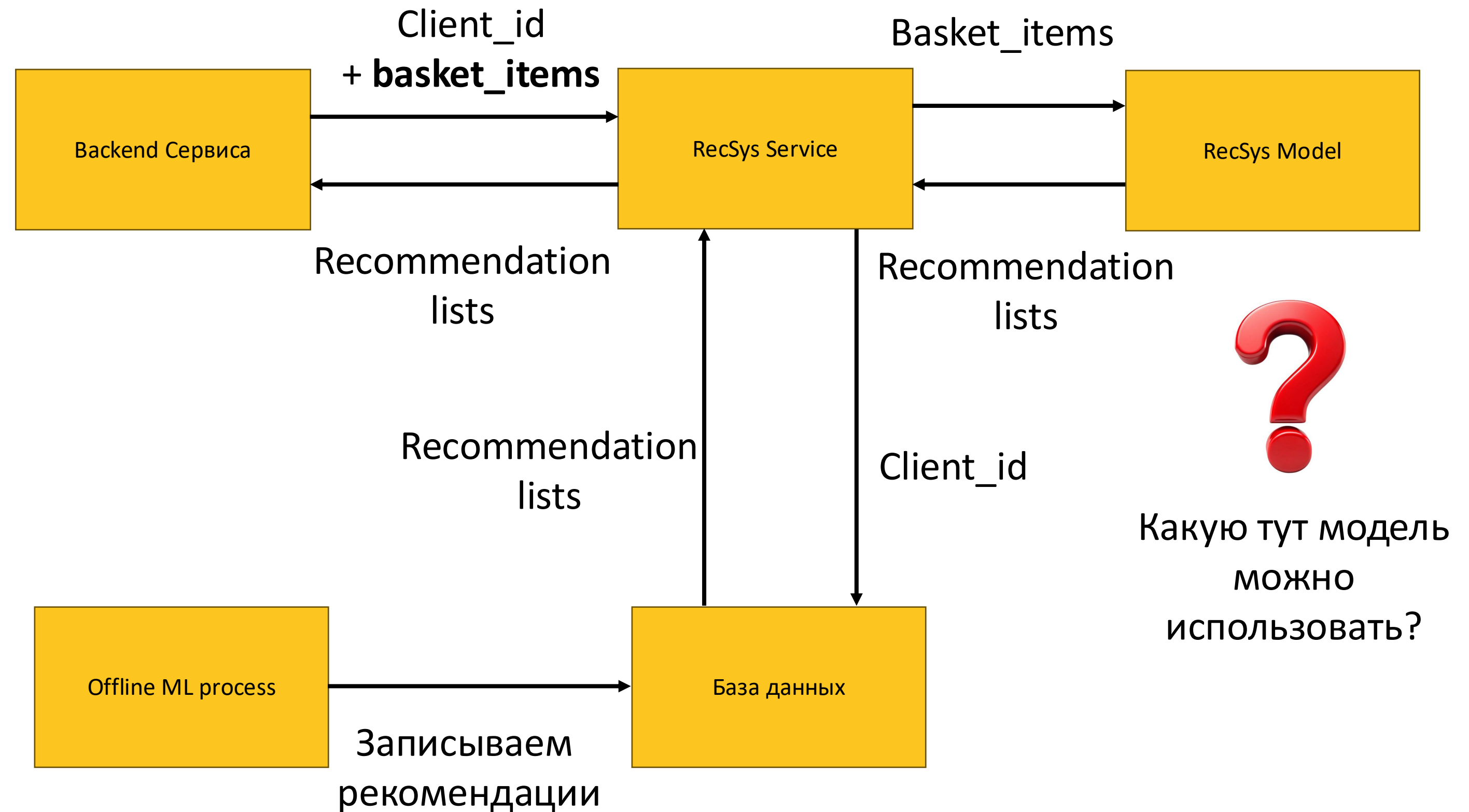
User_id	recommendations	Datetime_of_update
1	[1, 2, 3, 4, 5]	1345
2	[1, 12, 3, 4, 5]	1234
3	[1, 42, 12, 43]	2435

Как сделать полуперсональные рекомендации?

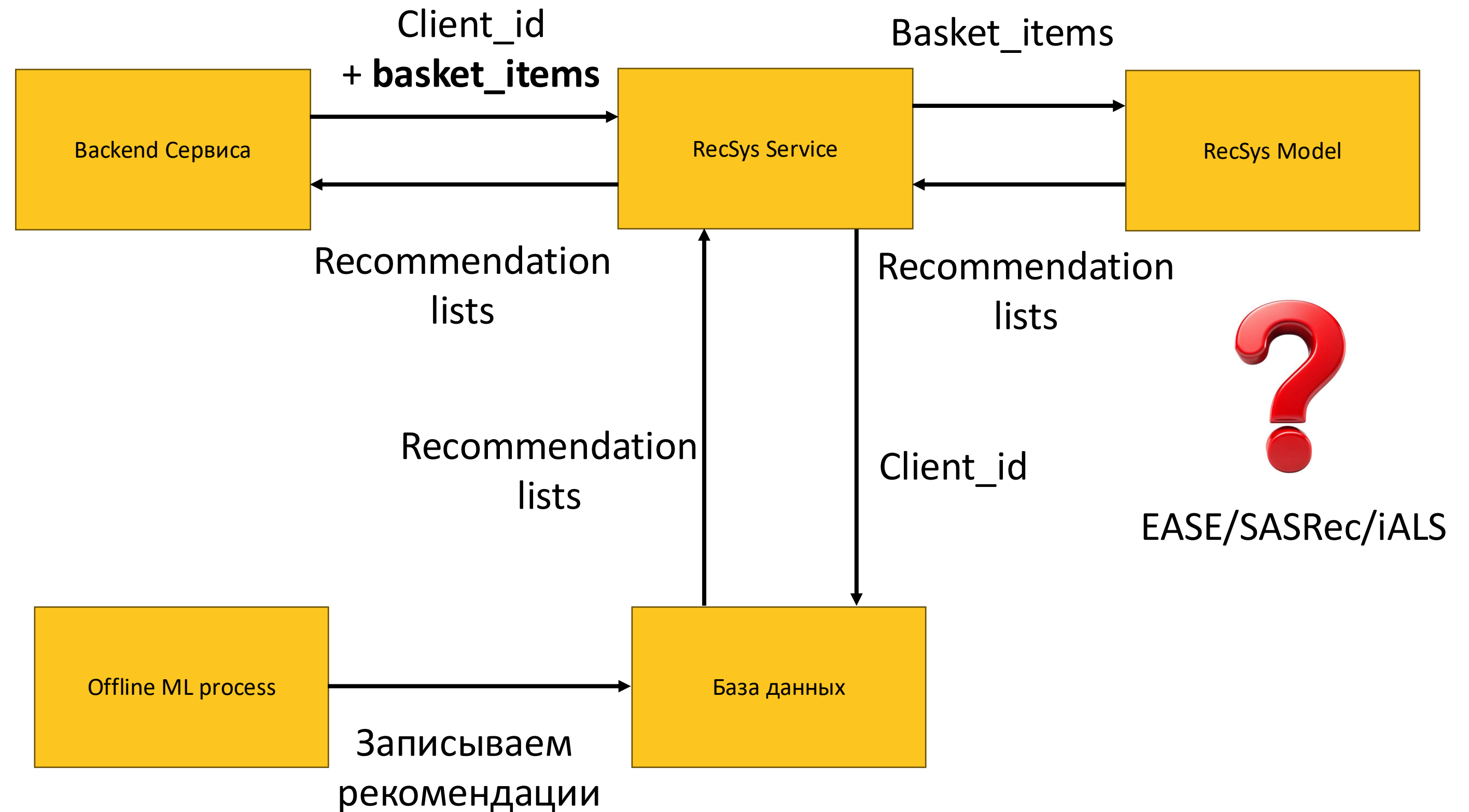
Идея: давайте сократим число списков хранимых, например:

- 1) Будем хранить вообще только N самых популярных списков рекомендаций (по топ-K)
- 2) Будем кластеризовать людей по соцдему, и хранить отдельно по ключам вида user-age-education-device, и для каждого ключа хранить популярный список рекомендаций
- 3) Кластеризуем списки рекомендаций и будем отдельно хранить user – cluster, и cluster – list

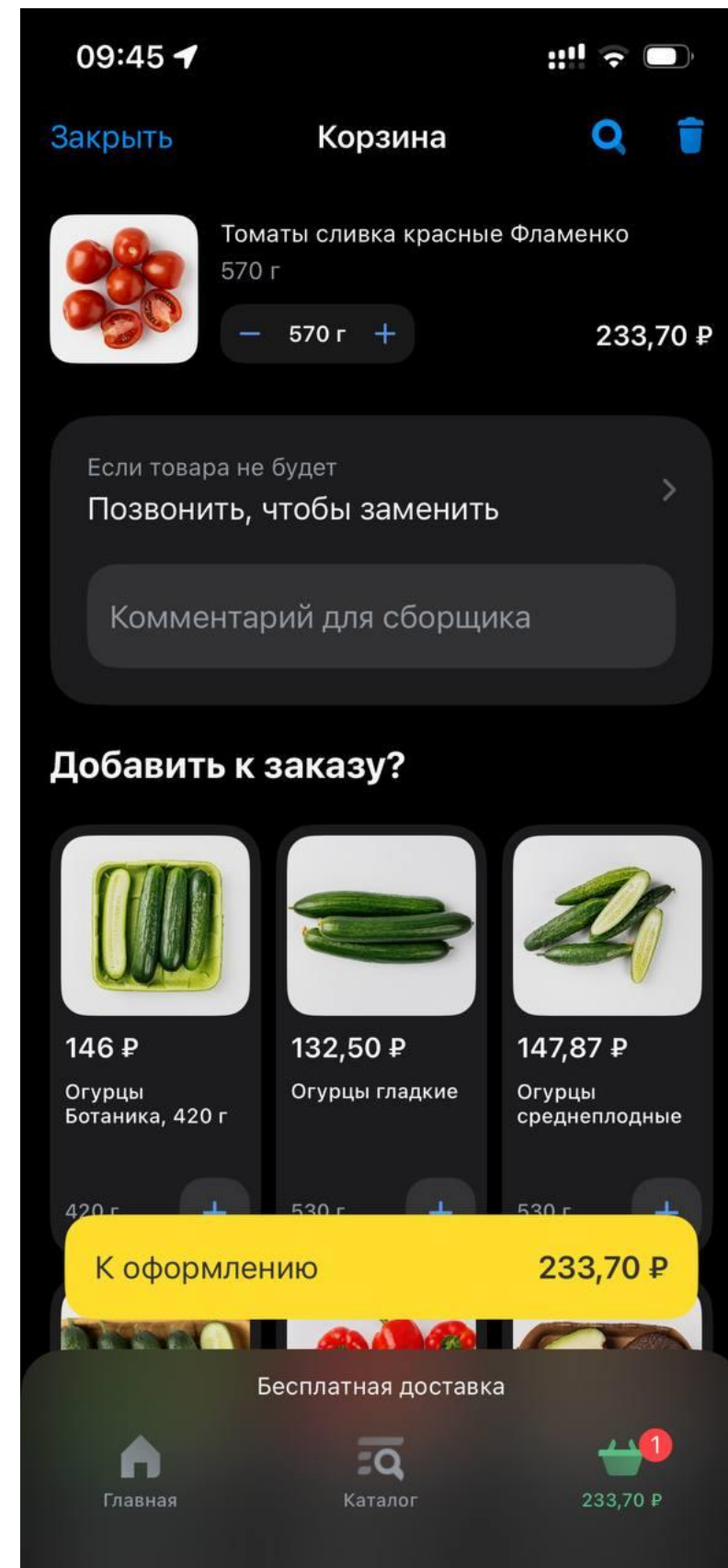
Передача текущего контекста



Передача текущего контекста



Как можно смешать выдачи рекомендаций?



Есть два списка рекомендаций, как их смешать?

1) Можно применить метод блендинга (смешивания списков):

- Для каждого айтема из списка считаем ранк вхождения айтема, суммируем их.
- Если айтем не входит во второй список, считаем его ранк top-K или другое значение
- Усредняем ранк. Чем меньше ранк, тем лучше

2) Можно сначала показать один список, а как только он заканчивается, показать сразу другой список (с удалением из него айтемов из первого списка)

3) Можно рассматривать это как кандидатогенераторы, и поверх списков применить еще отдельную модель

Применение EASE-like модели в онлайн

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

X

ITEMS * ITEMS

0	.	.	.	.	.	.	.	.
.	0	.	.	.	.	.	.	.
.	.	0	.	.	.	.	.	.
.	.	.	0	.	.	.	.	.
.	.	.	.	0	.	.	.	.
.	.	.	.	.	0	.	.	.
.	.	.	.	.	.	0	.	.
.	.	.	.	.	.	.	0	.
.	.	.	.	.	.	.	.	0

=

USERS * ITEMS

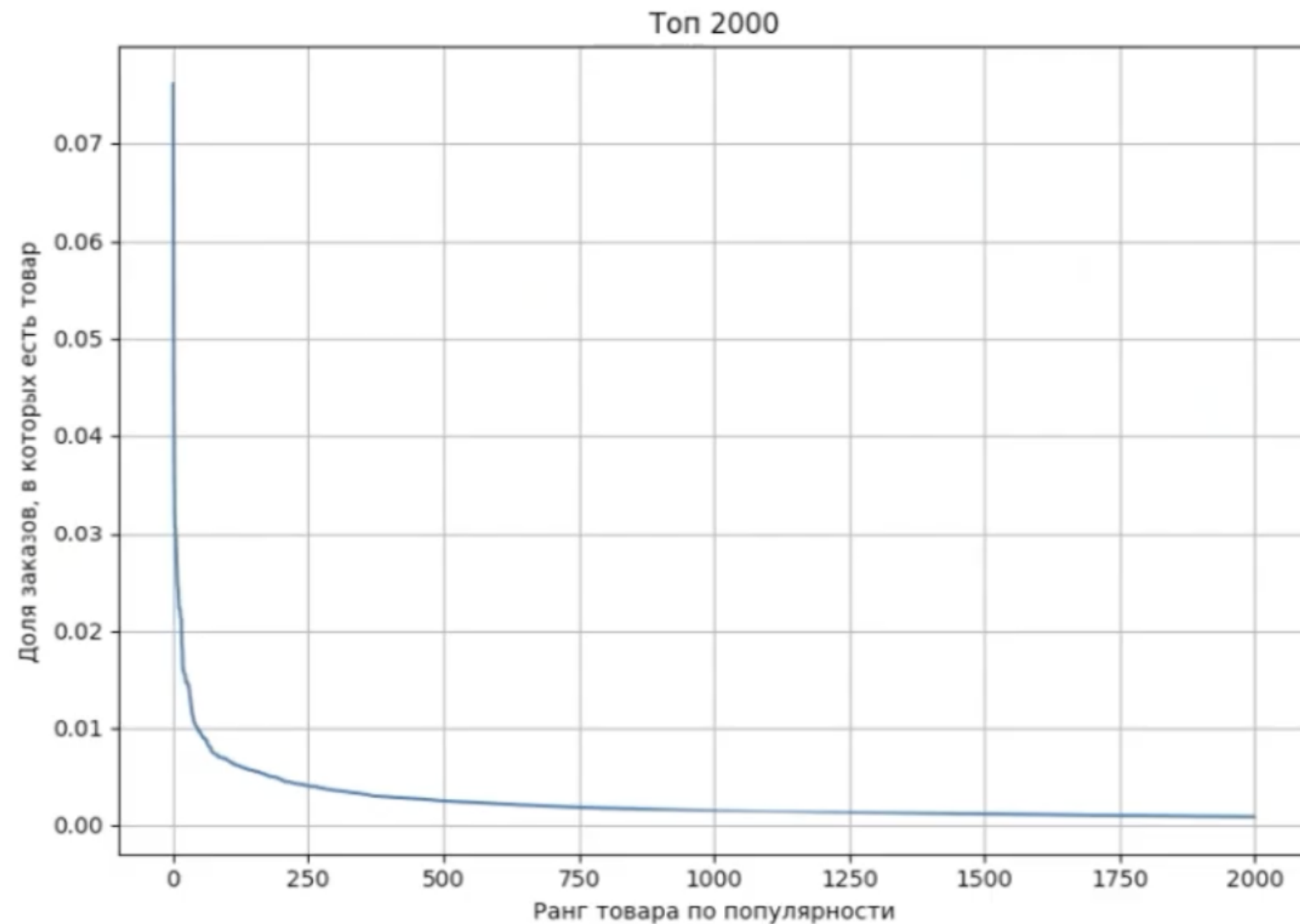
1			.	1				
			.	1		1		
1		1	.				1	1
			.		1			
	1		.					
			1					1
			.		1	1		
1		1	?				1	

В сервисе живет матрица item * item, ее можно хранить в памяти сервиса (отсюда ограничение на ее размер)

Можно брать только топ-к популярных айтемов, это жестко ограничит размер требуемой памяти

Все неизвестные айтемы для сервиса будут не учитываться.

Подбор top-k



Пример того, что в сервисе довольно быстро падает используемость товаров.

Полезно смотреть график «сколько интеракций покрывает top-K элементов»?

Например, топ-1000 айтеров покрывает 95% интеракций – можно взять топ-1000 и остальные не брать, вы потеряете только 5% интеракций

Инференс EASE

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

X

ITEMS * ITEMS

0	.	.	.	.	.	.	.	.
.	0	.	.	.	.	.	.	.
.	.	0	.	.	.	.	.	.
.	.	.	0	.	.	.	.	.
.	.	.	.	0	.	.	.	.
.	.	.	.	.	0	.	.	.
.	.	.	.	.	.	0	.	.
.	.	.	.	.	.	.	0	.
.	.	.	.	.	.	.	.	0

=

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1	?				1	

Нужно перемножить user_vector на матрицу item * item – наивная имплементация, но соответствует модели

Тут сложность $O(\text{item} * \text{item})$

Инференс EASE

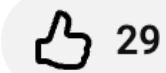
Онлайн up sell рекомендации (не ALS и не бустинг). Андрей Бабкин, Тинькофф

 Auto-dubbed



Желтый AI
2.95K subscribers

Subscribe



29



Share



Save



767 views 2 years ago #тинькофф #ai #ml

Можно ли строить рекомендации онлайн, не используя при этом ALS или бустинг? Андрей поделился, как у команды получилось это сделать для up sell рекомендаций в сервисе «Продукты» в Тинькофф Городе, с какими проблемами столкнулись и как их решили.
...more

Подробнее можно в докладе узнать, дальше будет саммари

Более оптимальный инференс EASE

➔ Аккумулируем строки, соответствующие элементам в корзине, из dense матрицы весов по одной в цикле

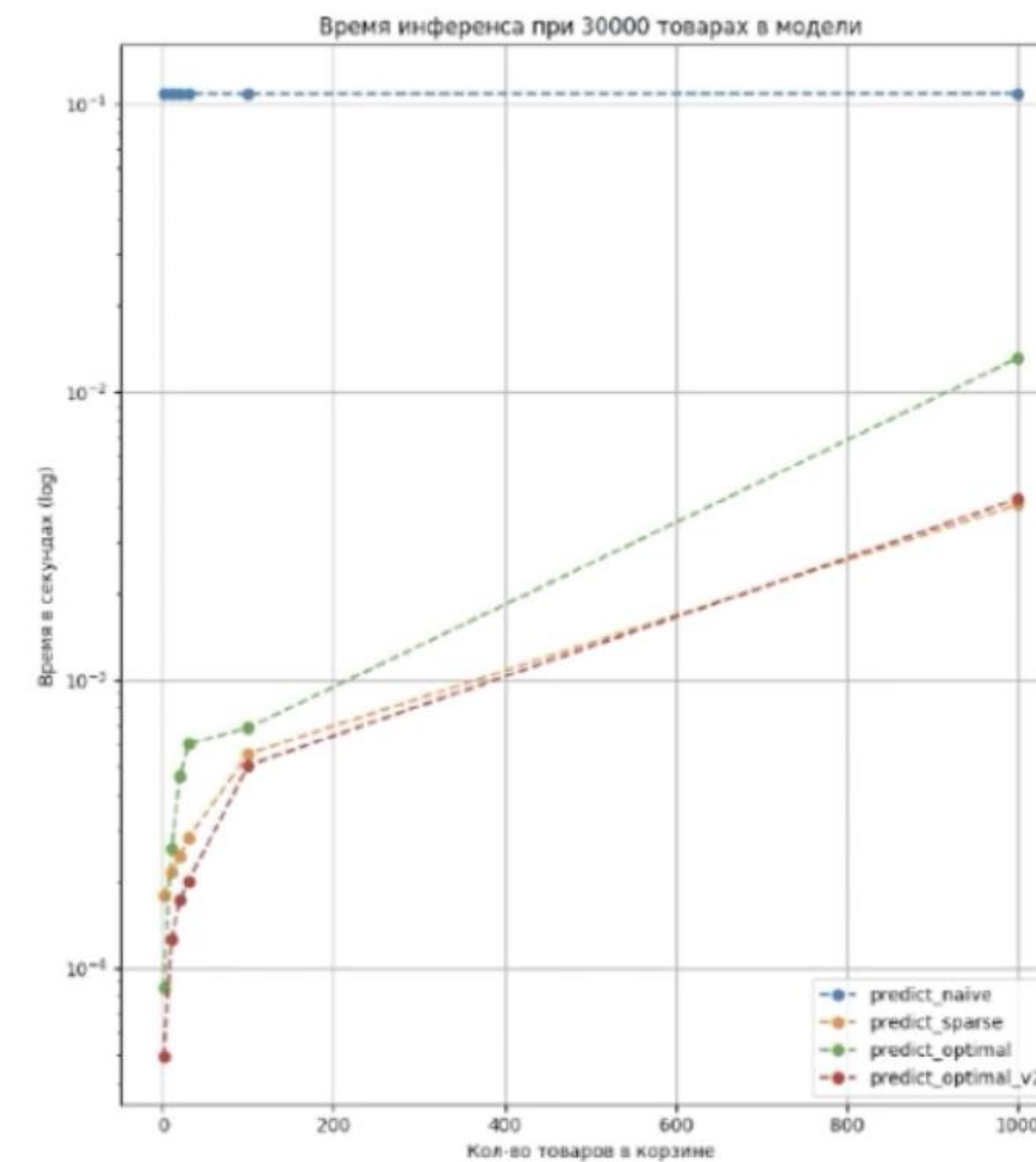
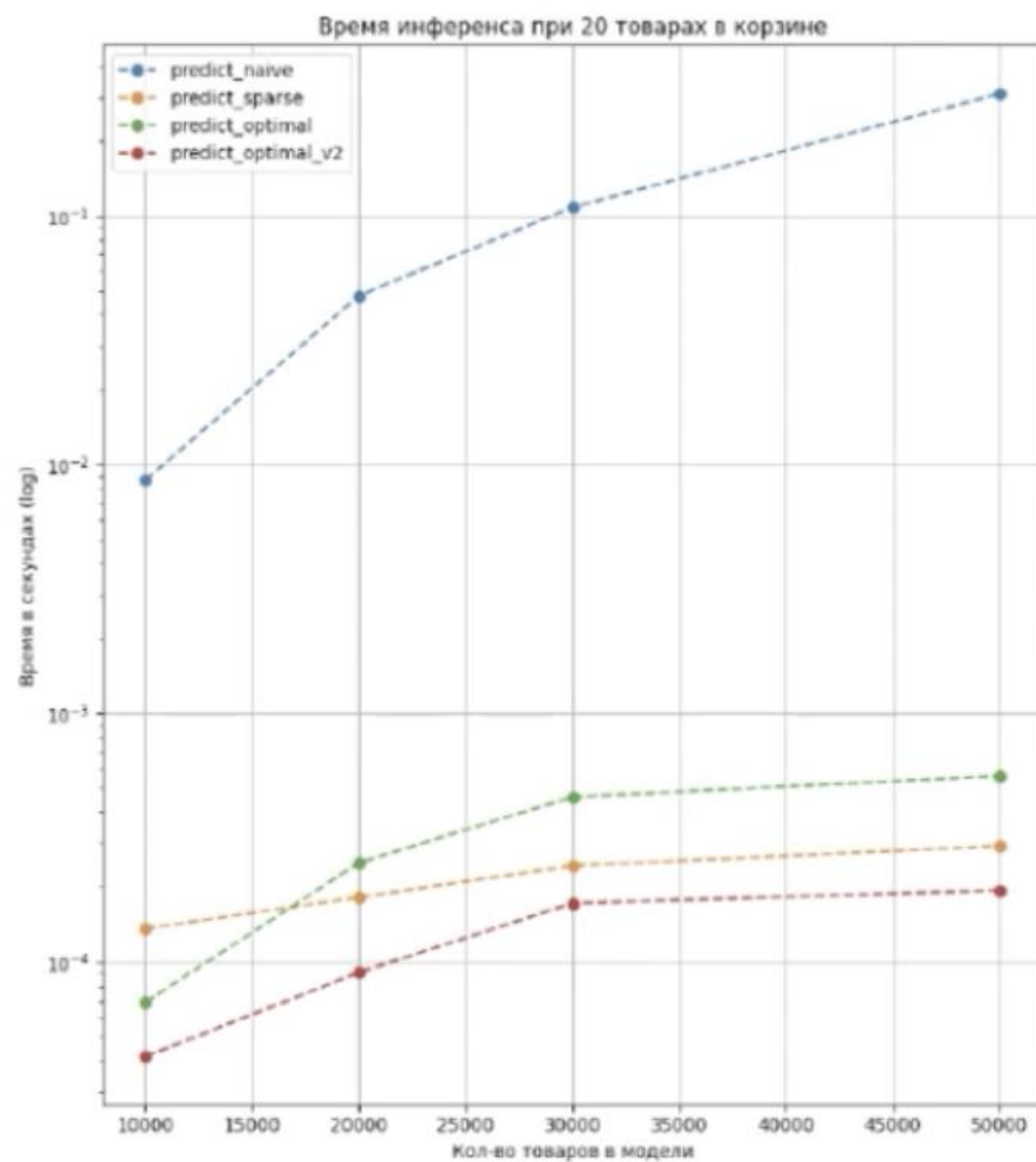
```
def predict_optimal_v2(
    weights: npt.NDArray[np.float32],
    items_id: list[int],
) -> npt.NDArray[np.float32]:
    scores = np.zeros(len(weights), dtype=np.float32)
    for item_id in items_id:
        scores += weights[item_id]
    return scores
```

Сложность $O(\text{cart_size} * I)$

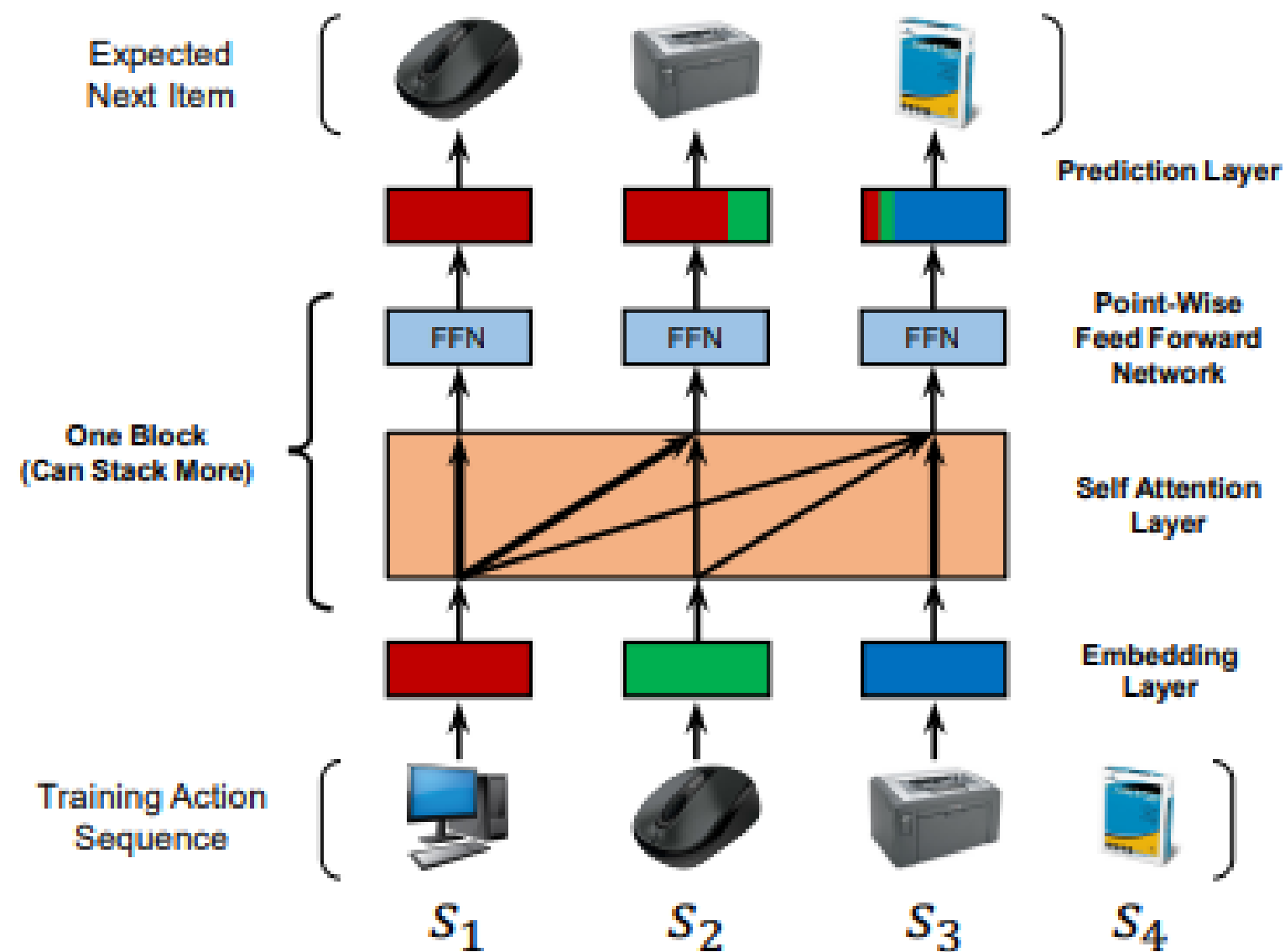
Сравнение имплементаций инференса

Время инференса при 30000 товарах в модели и 20 товарах в корзине

	mean	std	min	50%	95%	99%	max
predict_naive	0.108228	0.006909	0.105591	0.107324	0.111767	0.113958	0.318760
predict_sparse	0.000243	0.000064	0.000150	0.000234	0.000419	0.000500	0.000570
predict_optimal	0.000461	0.000090	0.000225	0.000479	0.000573	0.000616	0.000963
predict_optimal_v2	0.000172	0.000135	0.000074	0.000097	0.000511	0.000563	0.000626



Инференс SASRec



Мы можем брать `basket_ids` и применять инференс. В чем тут сложности?

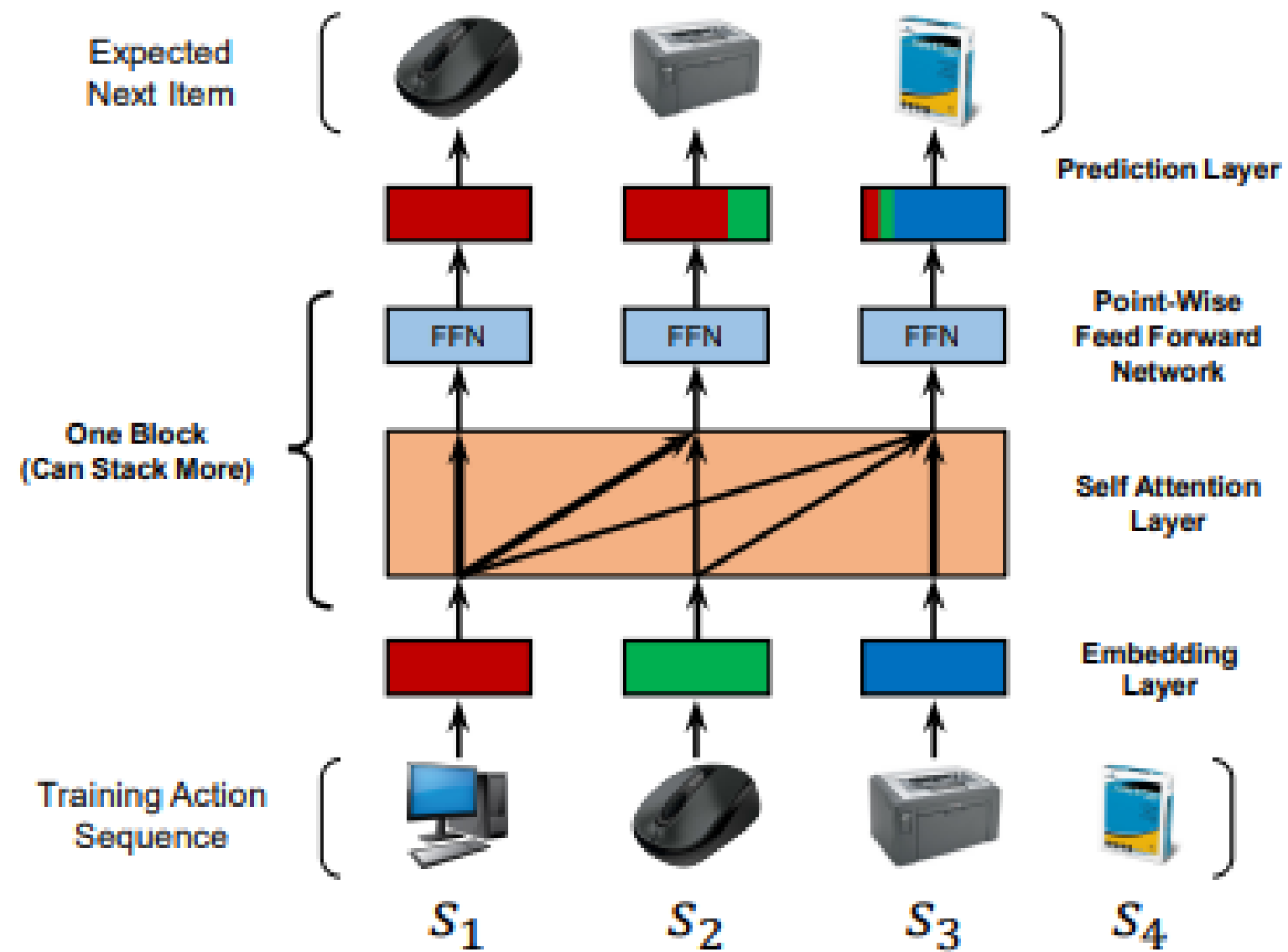
1) Длинные последовательности. Квадратичная зависимость инференса от длины последовательности

- Можно делать обрезание последовательности
- Можно снижать параметры трансформера (число слоев, голов, размерность модели)

2) Может быть много айтемов. Мы можем использовать ANN, получив `user_vector`, потом через ann найти ближайшие items

3) Также само время ответа зависит от размера трансформера, необязательно он должен быть большим

Инференс SASRec в онлайн из нашей практики

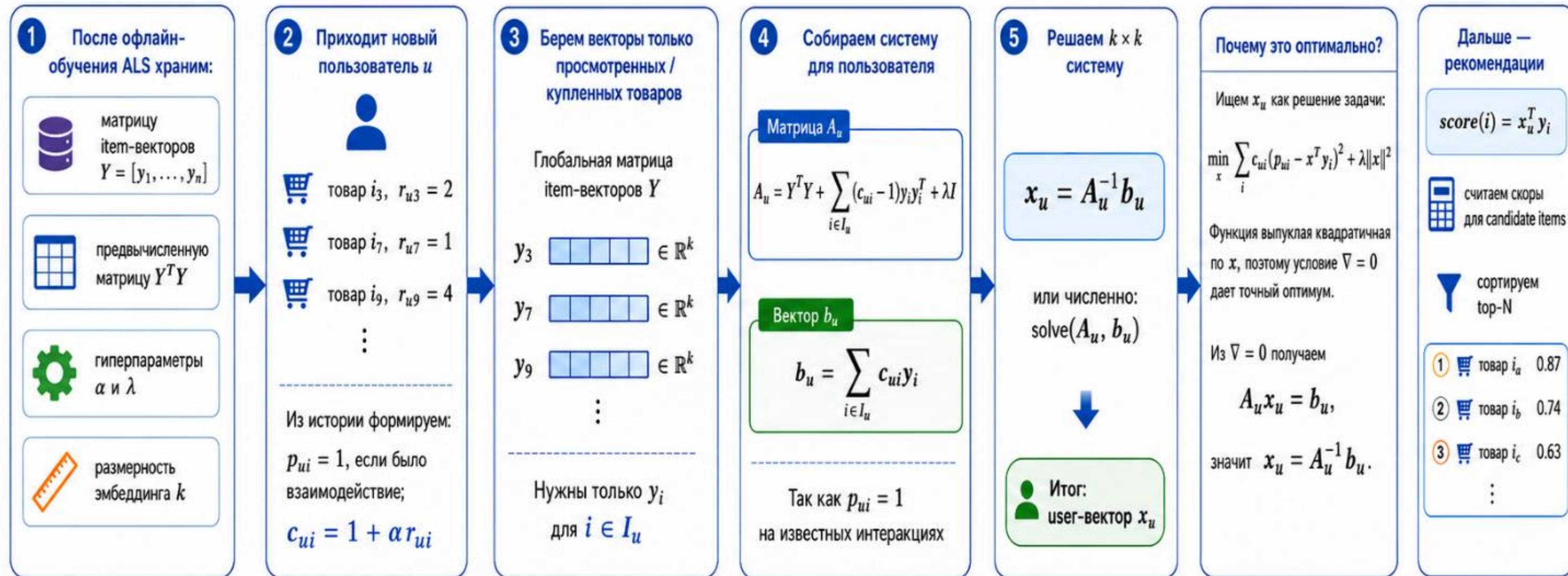


- 100 событий пользователя последние
- 100к айтеров
- 3-5 rps (запросов в секунду) (макс 15 при такой конфигурации)
- Blocks/heads/dim 4/4/512
- 0.5 cpu + 4 GB RAM, **gpu нет**
- Dot product по всем айтерам, топ-200 забираем
- Инференс в онлайн
- Нет ANN инференса

Такое обрабатывает за 50 мс

Инференс iALS

Получение вектора нового пользователя в iALS "на лету"



Ключевая идея: переобучать ALS не нужно — для нового пользователя решаем только одну маленькую линейную систему размерности k , используя уже обученные item-векторы.

I_u — множество товаров, с которыми пользователь u взаимодействовал
 $c_{ui} = 1 + \alpha r_{ui}$ — степень уверенности
 k — размерность эмбединга (обычно 32...256)

Обогащение item_id

Item_id не показываются пользователям напрямую, нужно некоторое обогащение:

- 1) Найти картинку
- 2) Найти категорию
- 3) Найти цену
- 4) Найти аудио/видео содержание
- 5) Сделать еще бизнес логику (например, фильтрацию, но про это отдельно).

Это все занимает время и ресурс, каждая операция! Чем меньше делаем, тем проще и быстрее

Такие обогащения могут создавать нагрузки в другие системы, что тоже не очень хорошо.

Пагинация

Пагинация – разделение выдачи на "pages"

Идея: не забирать и отрисовывать сразу 1000 элементов, а делать это пачками (например, по 10 элементов).

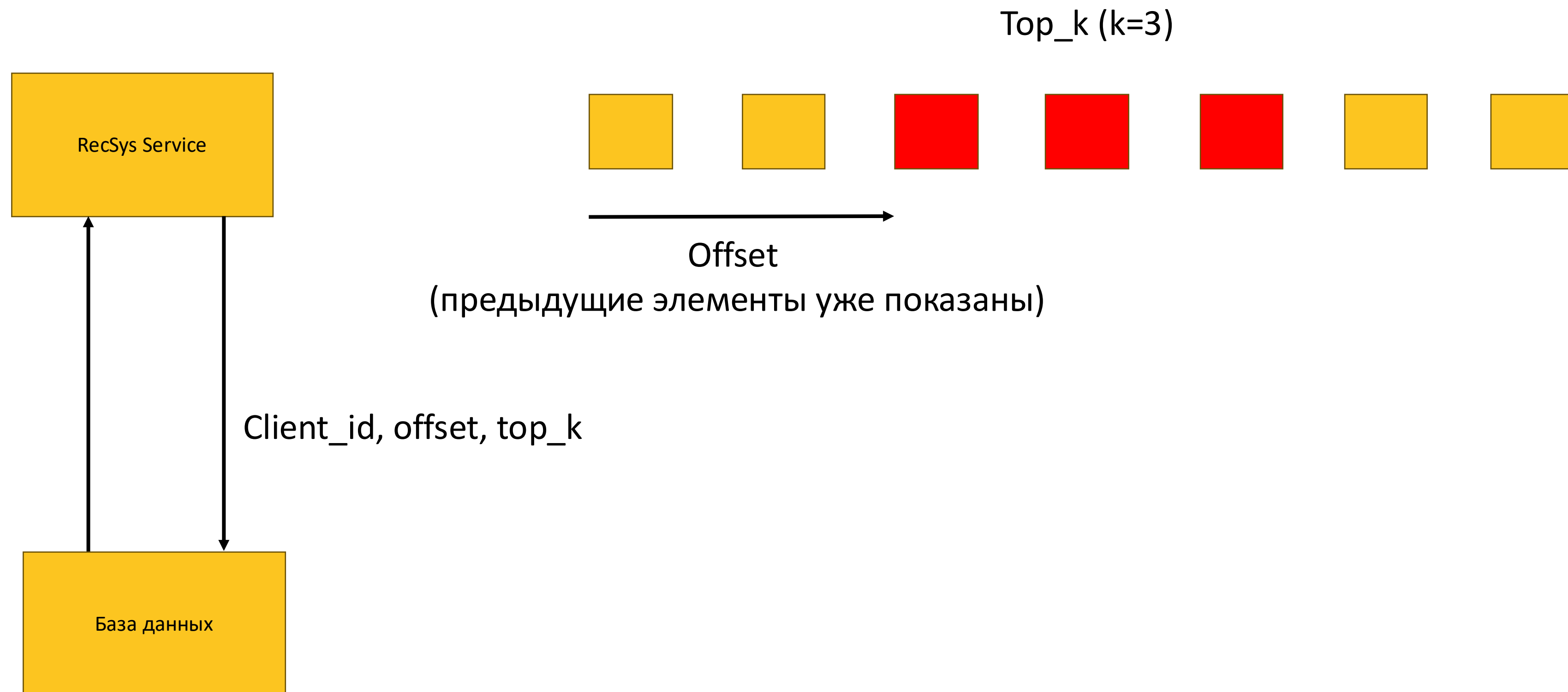
Плюсы:

- 1) Мы не собираем информацию по айтемам, которые пользователь не посмотрит (Не нагружаем сервисы)
- 2) Мы не тратим время операции выше и отвечаем быстрее
- 3) Мы снижаем шанс, что что-то пойдет не так
- 4) Мы оставляем возможность перестроиться под действия пользователя в ленте

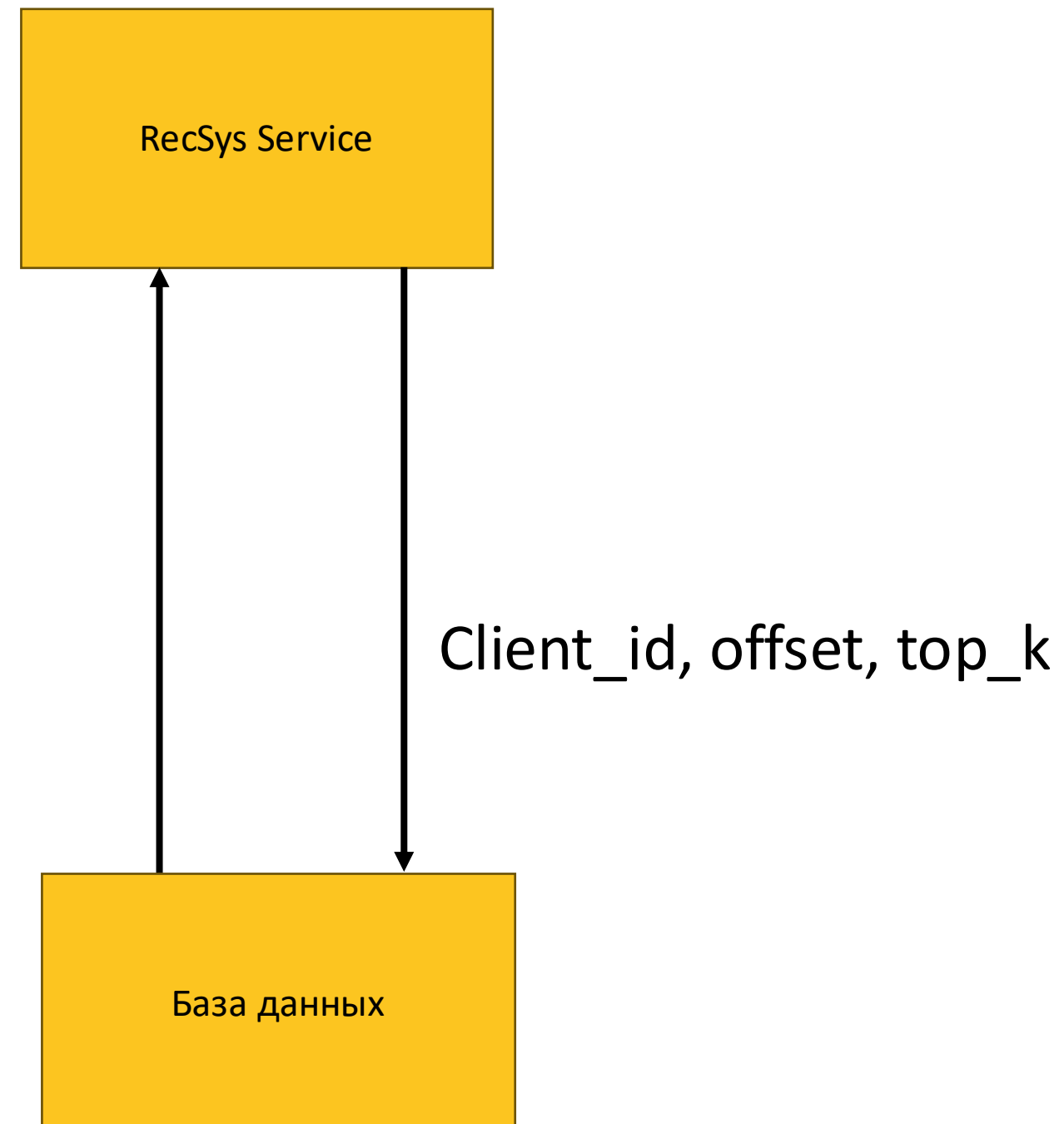
Минусы:

- 1) Более сложная реализация

Пагинация



Пагинация и донастройка рекомендаций



- 1) Запросили и показали 10 айтемов
- 2) Пользователь кликнул на айтем на 9 позиции
- 3) Мы можем учесть этот клик для пересчета рекомендаций
- 4) Запросили следующие 10 айтемов. Если п. 3 сделан, они учитывают действия в п. 3. Персонализация лучше

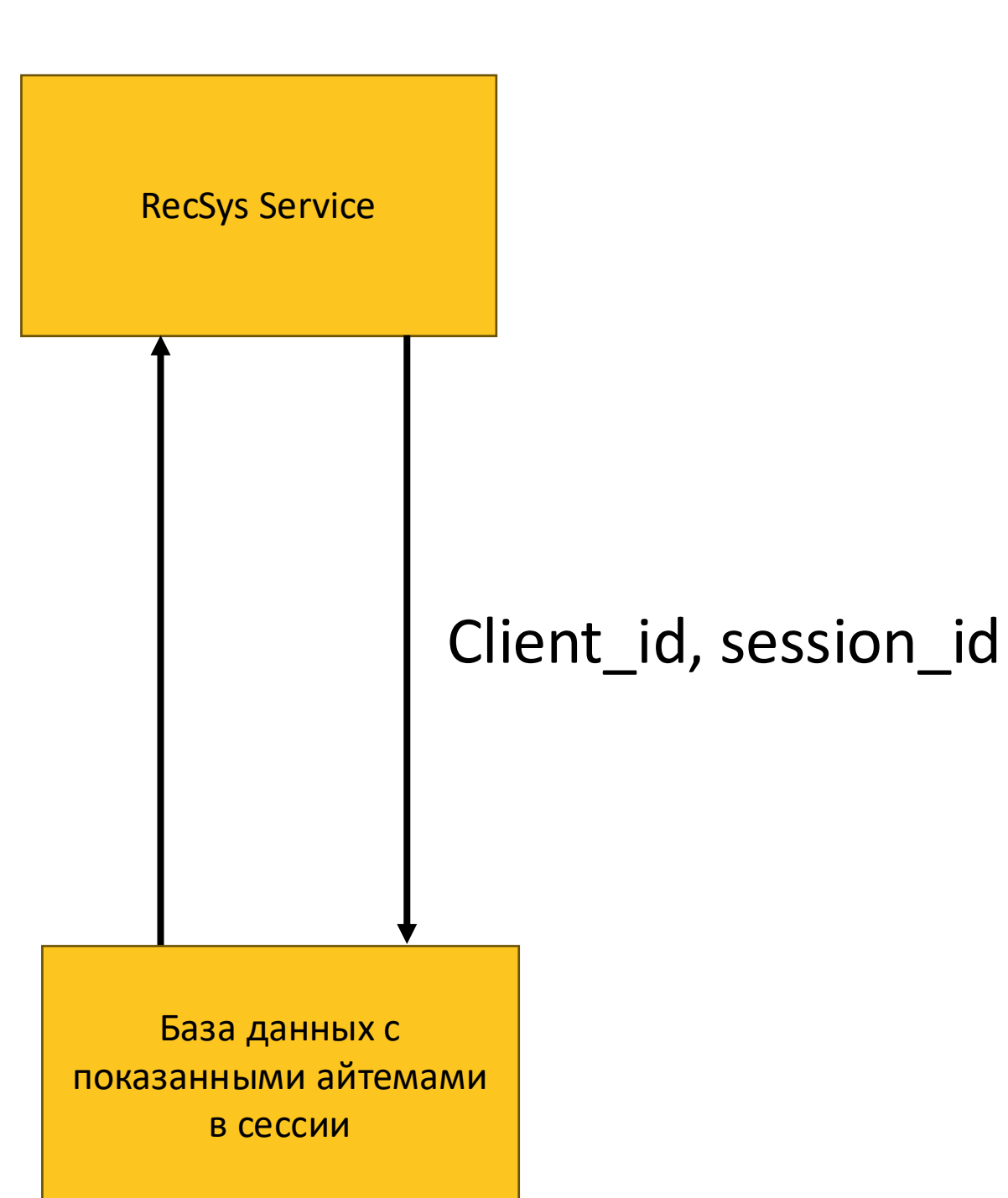
Пагинация и донастройка рекомендаций



Пагинация и донастройка рекомендаций



Пагинация и донастройка рекомендаций



Мы можем отдельно хранить все просмотренные айтемы юзером.

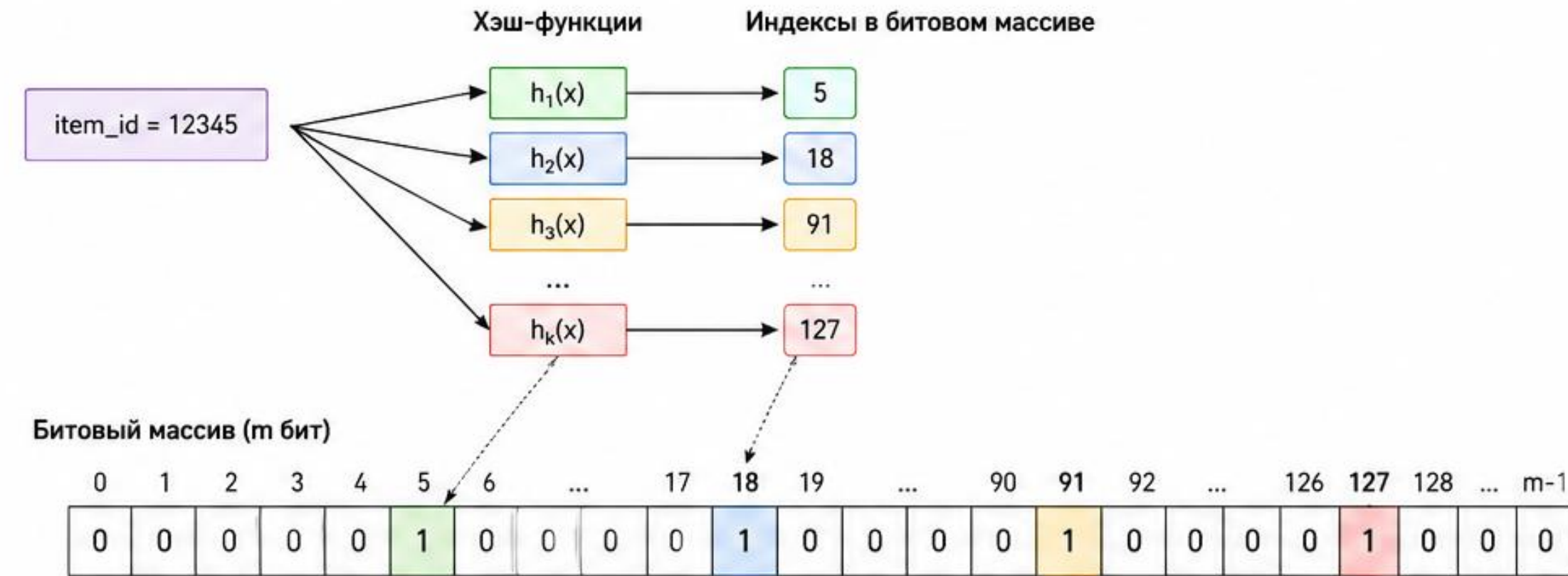
Тогда для каждого нового айтема проверим его на вхождение в список.

Важно – надо определить условие сброса этого списка, например, сессия, или 1 день, или еще что-то.

Время проверки – айтема $O(i) - 1$ – длина просмотренных айтемов

Схема работы Bloom Filter

1) Добавление item_id в Bloom Filter пользователя



Примеры хэш-функций

1) $h_1(x) = ((a_1 * x + b_1) \bmod P) \bmod m$
где $a_1=17$, $b_1=43$, $P=1000000007$, m = размер битового массива

2) $h_2(x) = ((a_2 * x + b_2) \bmod P) \bmod m$
где $a_2=31$, $b_2=97$, $P=1000000007$, m = размер битового массива

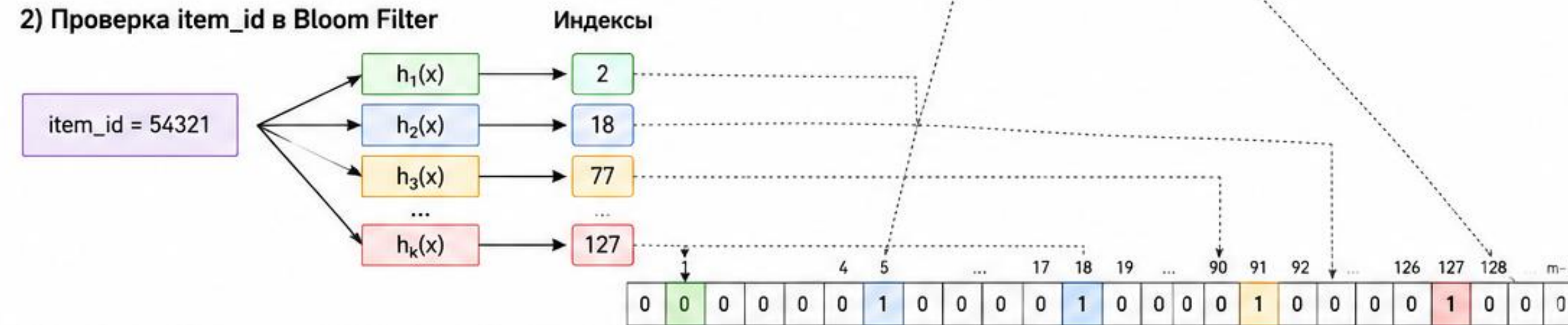
3) $h_3(x) = ((a_3 * x + b_3) \bmod P) \bmod m$
где $a_3=59$, $b_3=11$, $P=1000000007$, m = размер битового массива

...

k) $h_k(x) = ((a_k * x + b_k) \bmod P) \bmod m$
где a_k и b_k – другие константы

m – размер битового массива (например, 1024 бита)
 k – количество хэш-функций (например, 7)

2) Проверка item_id в Bloom Filter



Проверяем все биты:

- Если хотя бы один бит = 0 → item **ТОЧНО** не был добавлен
- Если все биты = 1 → item **ВОЗМОЖНО** был добавлен (из-за коллизий)

Состояние фильтра актуально для конкретного пользователя



Проверка на этапе рекомендаций (serving time)

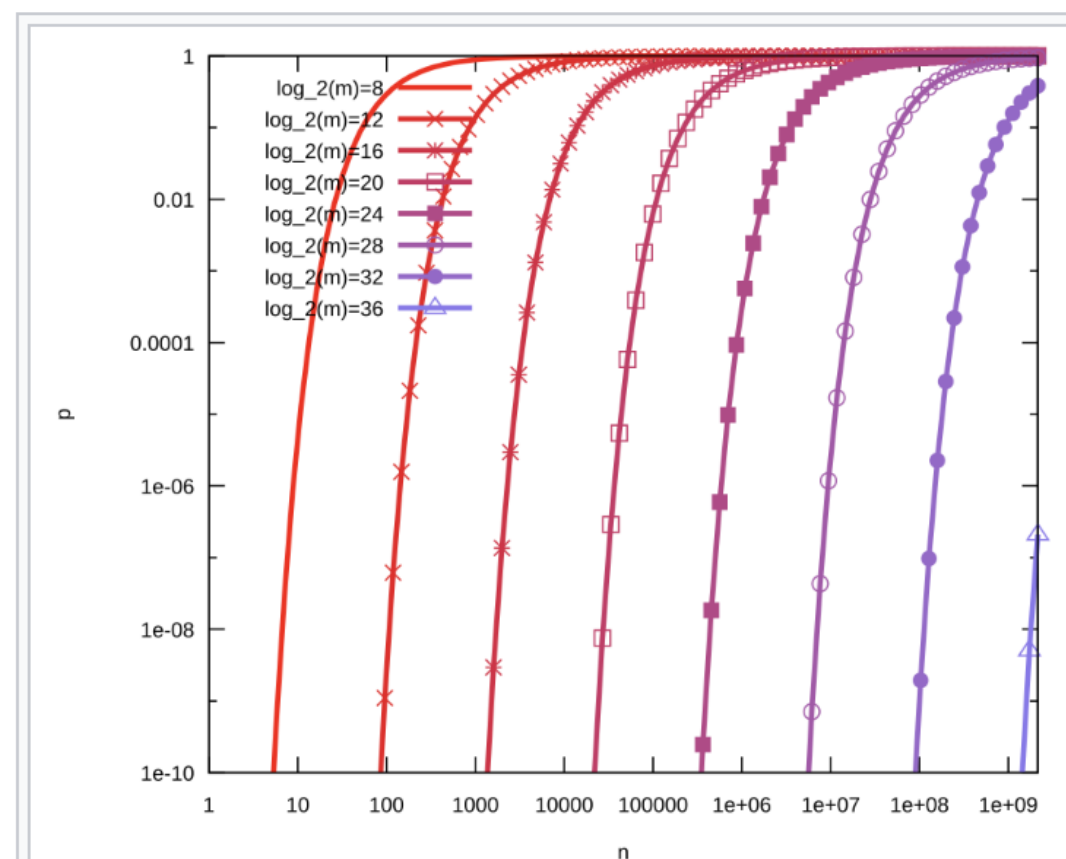
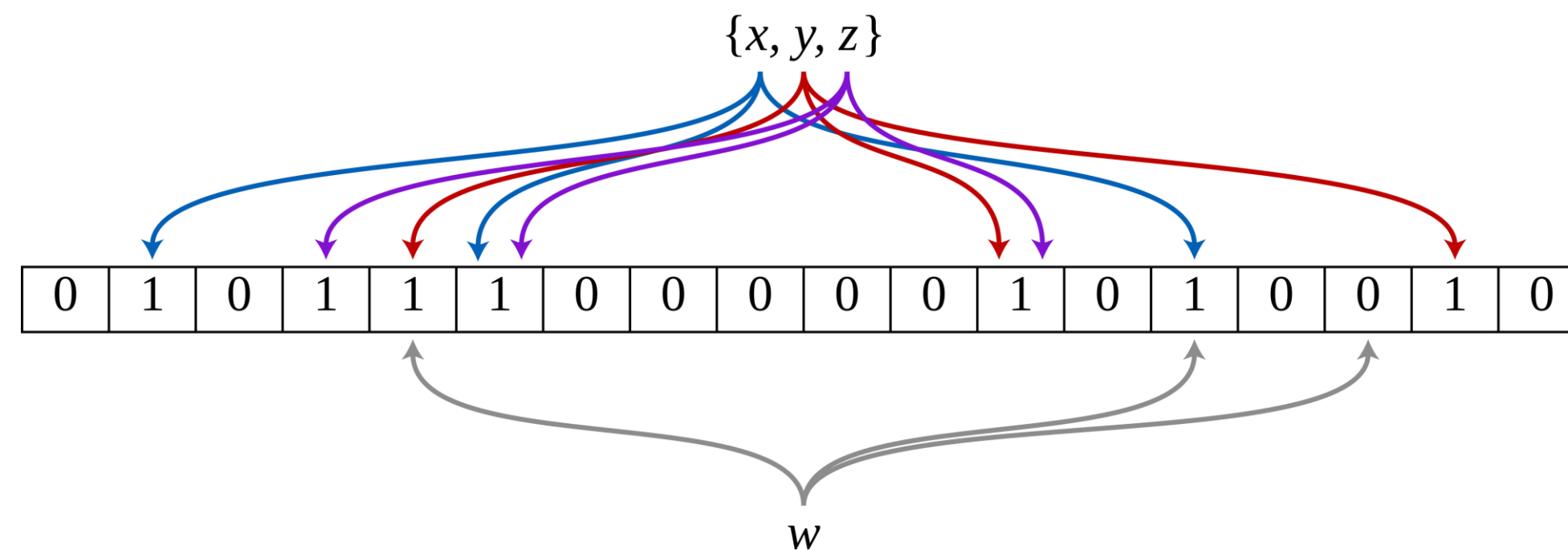
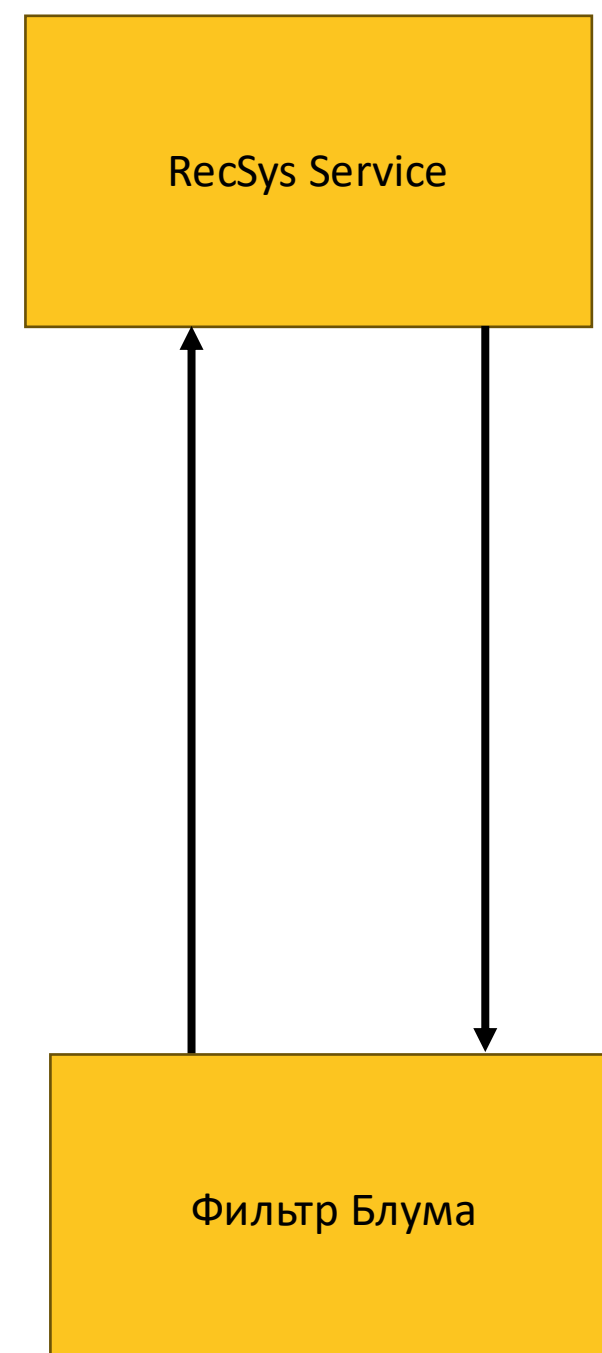
Запрос: user_id = А, item_id = 77777

Проверяем в Bloom Filter пользователя А

Если все биты = 1 → item возможно уже был просмотрен (не показываем)

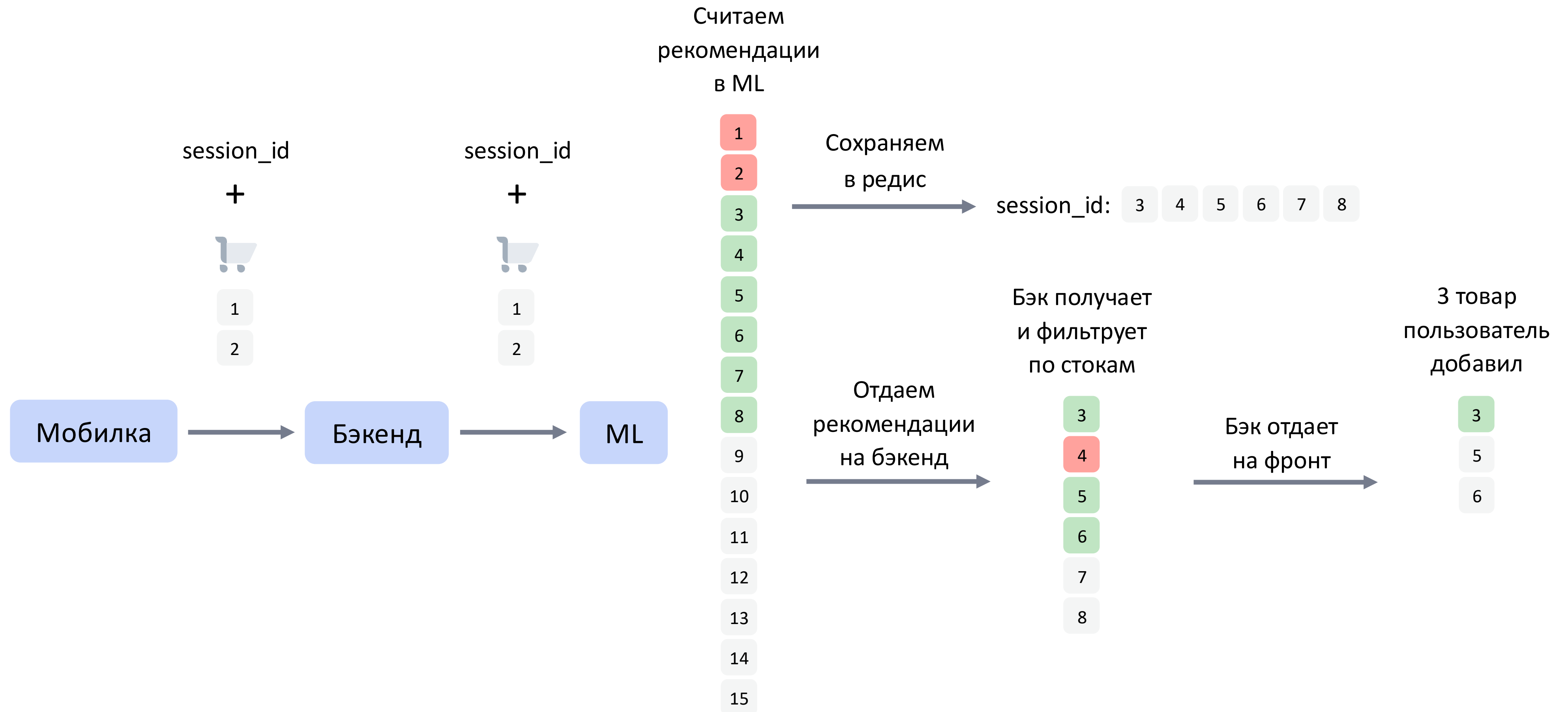
Если есть хотя бы один бит = 0 → item точно не был просмотрен (можно показывать)

Пагинация и донастройка рекомендаций

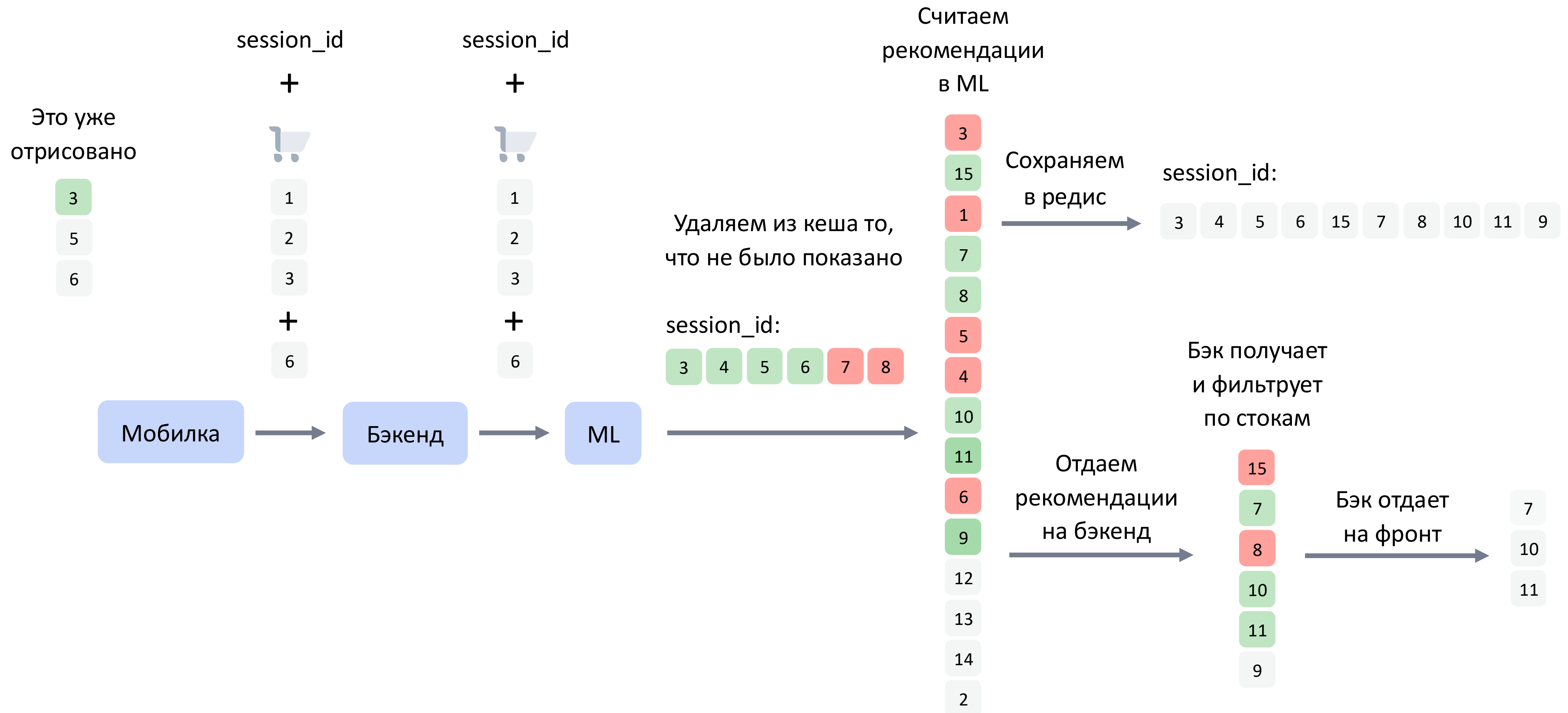


Оценка вероятности ложного срабатывания p как функция от числа вставленных элементов n и размера битового массива m , при оптимальном числе хеш-функций $k = (m/n) \ln 2$.

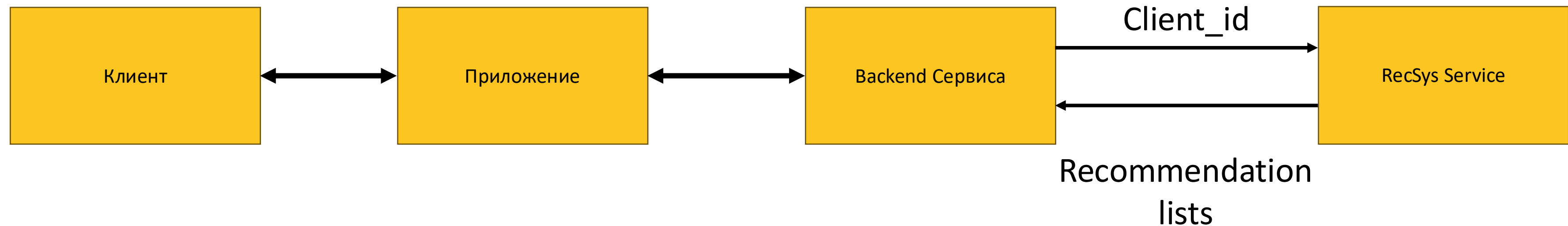
Пример схемы с пересчетом и пагинацией



Пример схемы с пересчетом и пагинацией

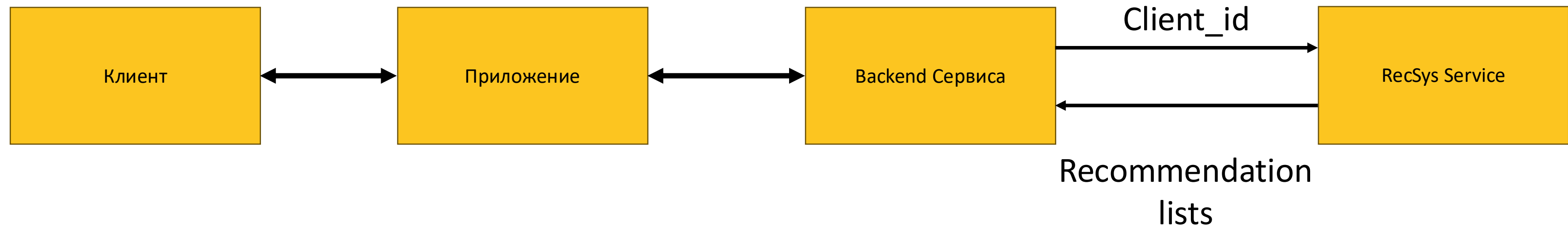


Метрики для мониторинга оффлайн сервиса



- 1) Какая доля пользователей, у которых посчитаны рекомендации? А сколько юзеров не было в таблице? Должно быть 1
- 2) "Возраст" рекомендаций = время запроса – время записи в БД. Должен стремиться к 0
- 3) Задержка учета данных = время запроса – время последнего учтенного события (может считаться по разным критериям). Должна стремиться к 0

Метрики для мониторинга оффлайн сервиса

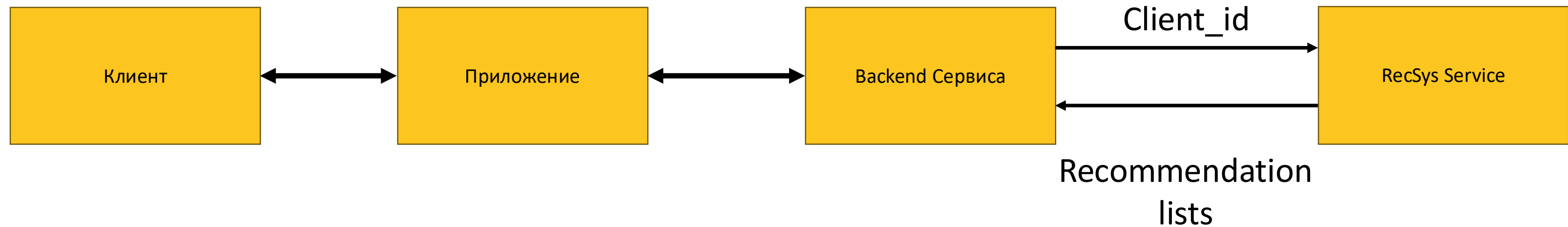


В оффлайн процессе есть джобы по крону, которые обсчитывают всю базу. Например, раз в 1 день.

Тогда источники задержки:

- 1) Периодичность запуска
- 2) Задержка загрузки данных в хранилище (может быть тоже +1 день)
- 3) Задержка самого расчета. Если считается 12 часов, то это все тоже затягивает
- 4) Задержка в загрузке рекомендаций в БД.

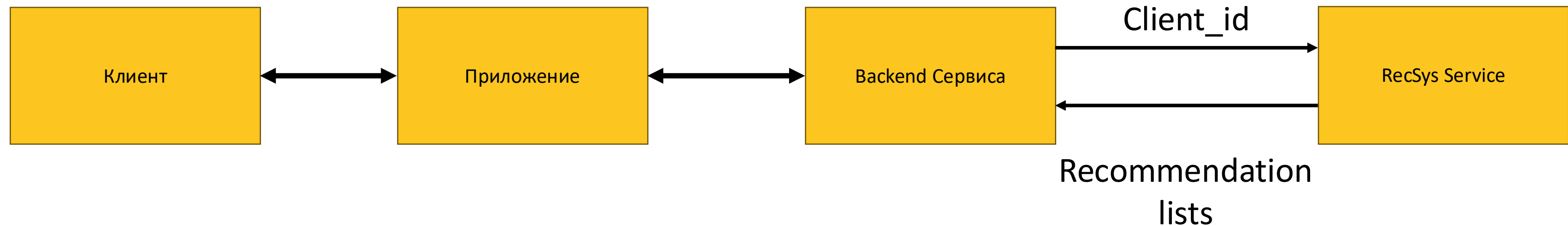
Очередь круговых апдейтов



Пусть есть много клиентов.

- 1) Для каждого клиента сделаем поле `last_update`.
- 2) Будем запускать каждые N часов большие батчи на k клиентов, отсортированными по `last_update`.
Внутри обрабатываем клиентов по маленькому батч сайзу
- 3) Клиентов с апдейтом ставим в конец очереди, переписывая их `last_update` на `current time`

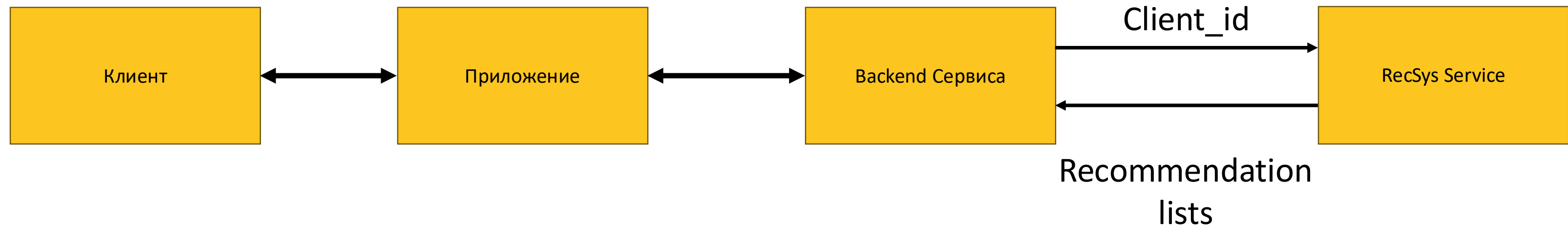
Задержка в оффлайне



Есть два варианта:

- 1) Обсчитываем маленький батч юзеров (например, 1к юзеров), сразу отправляем в БД рекомендации
 - 1) Плюсы:
 - 1) Сокращаем время задержки учета данных
 - 2) В случае поломки процесса часть юзеров обновится
 - 2) Минусы:
 - 1) Нецелевое использование ресурсов. Если инференс на гпу/много RAM/CPU, то отправка скоров – не гпу процесс -> неоптимально.
- 1) Обсчитываем большой батч юзеров (например, 1М юзеров) и только потом шлем сразу всех в БД. Тут плюсы и минусы противоположны п. 1

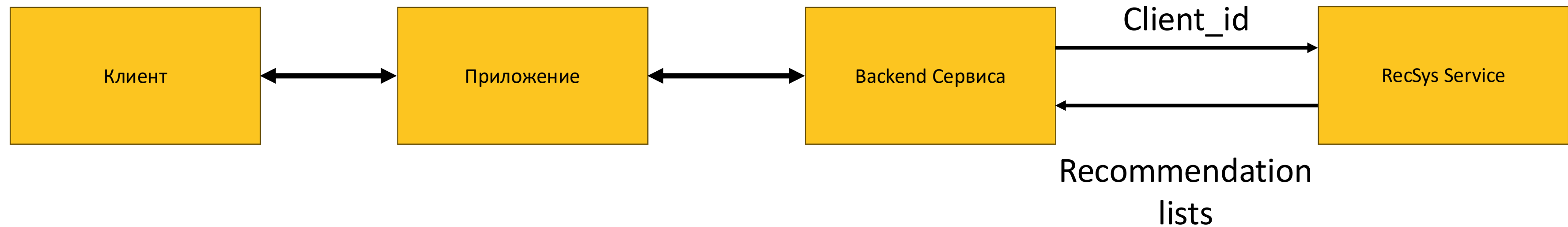
Можно сделать лучше



Как сократить средний возраст рекомендаций и учета данных в такой схеме, доработав ее?



Recent inference



Гипотеза: по активным юзерам есть новые действия, и если они еще раз зайдут, по ним важнее быстрее учесть обновления

Давайте по некоторым юзерам обновлять рекомендации быстрее

Recent inference

Пусть есть много клиентов.

- 1) Для каждого клиента сделаем поле `last_update`.
- 2) Будем запускать каждые N часов большие батчи на k клиентов, отсортированными по `last_update`.
Внутри обрабатываем клиентов по маленькому батч сайзу
- 3) Клиентов с апдейтом ставим в конец очереди, переписывая их `last_update` на `current time`
- 4) **Каждые P минут будем собирать юзеров, которые сделали что-то за последнее время, обновим по ним все данные**
- 5) **Запустим такую же джобу, которая обновит рекомендации только по этим юзерам**
- 6) **Самые активные юзеры получают обновление быстрее. Когда они зайдут еще раз, по ним будут более свежие рекомендации с учетом их недавних действий. Возраст рекомендаций и задержка учета событий упадет**

Recent inference: нюансы

- 1) Важно реализовать схему, чтобы «большой инференс» с более старыми данными не перезагружал свои рекомендации более свежие
- 2) В идеале, чтобы recent_inference и глобальный инференс были с почти идентичными реализациями, чтобы не дублировать всю логику
- 3) Триггер для срабатывания recent_inference может быть разный.

1) Пользователь кликнул на айтем – средний сигнал, пусть таких 10к юзеров за период

2) Пользователь увидел айтем - слабый сигнал, пусть таких юзеров 100к за период

Trade off: берем больше юзеров, recent inference работает дольше, сигналы слабее

Может быть несколько recent_inference процессов: быстрый для одного типа сигнала, медленный для другого

Recent inference – это эвристика

Если пользователи возвращаются в сервис реже, чем срабатывает глобальный апдейт – recent_inference бесполезен

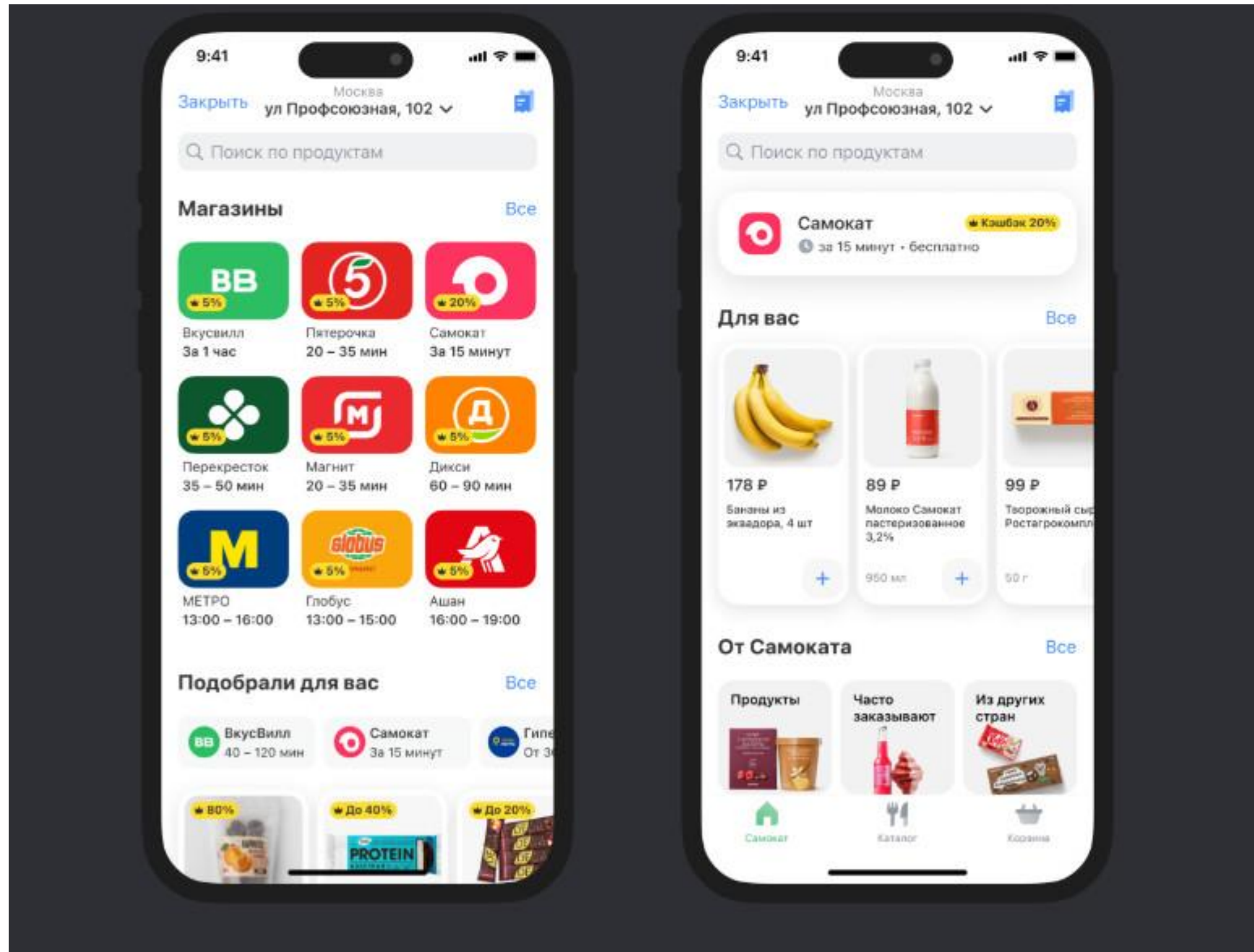
В общем случае есть некоторый бюджет (мощности) на пересчет рекомендаций, мы хотим:

- 1) Максимально снизить возраст рекомендаций
- 2) Сделать не так много бесполезных пересчетов

В идеале, должна быть

- 1) модель предсказания захода пользователя в следующий момент времени
- 2) Процесс, который под заход пользователя максимально обновит ему рекомендации

Проблема оффлайн списков с контекстом

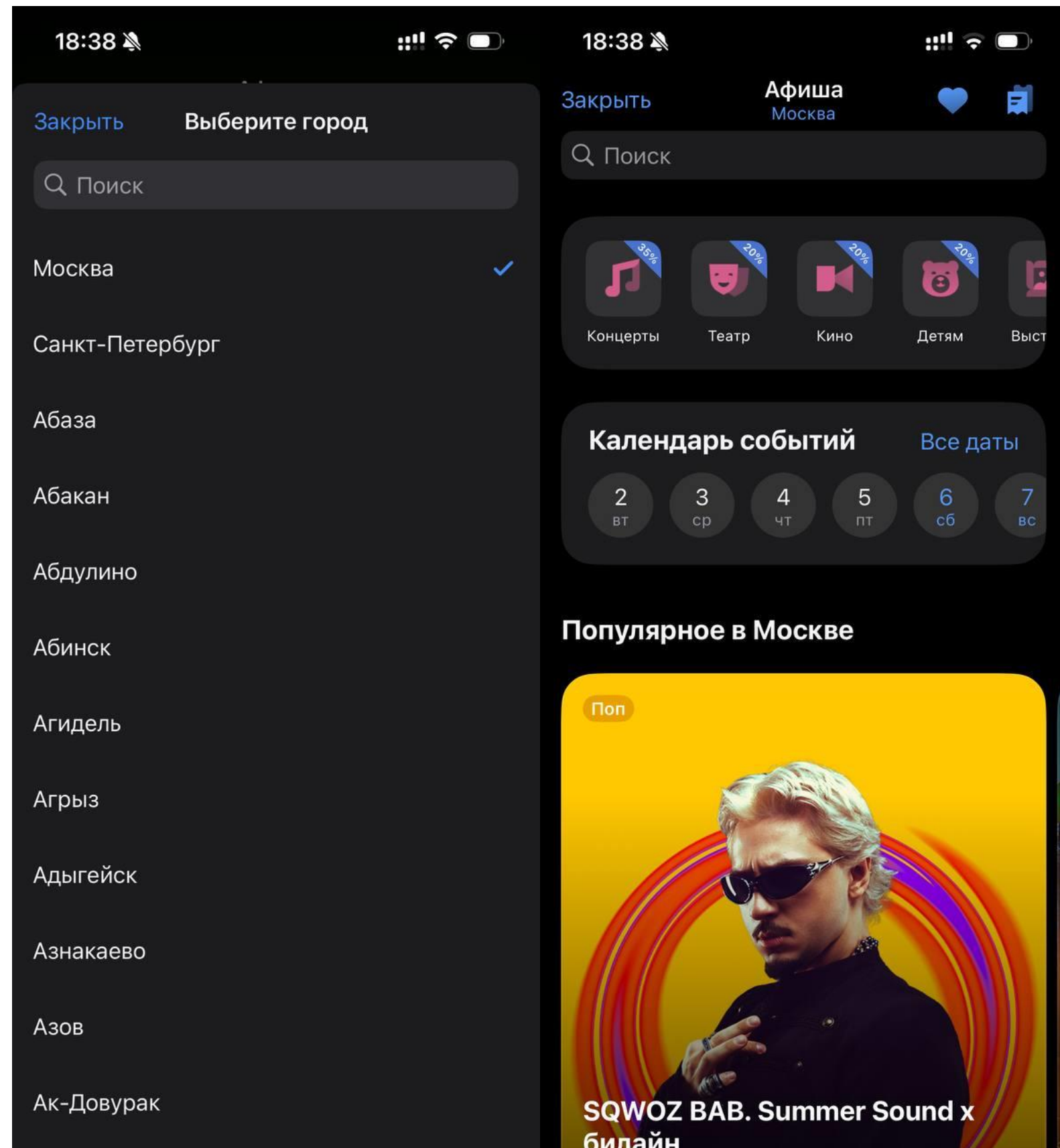


Есть много разных магазинов, и мы должны для каждой пары user – магазин насчитать оффлайн списки.

Мы не знаем заранее, в каком магазине купит пользователь.

Для непопулярных магазинов еще меньше пользы

Проблема оффлайн списков с контекстом

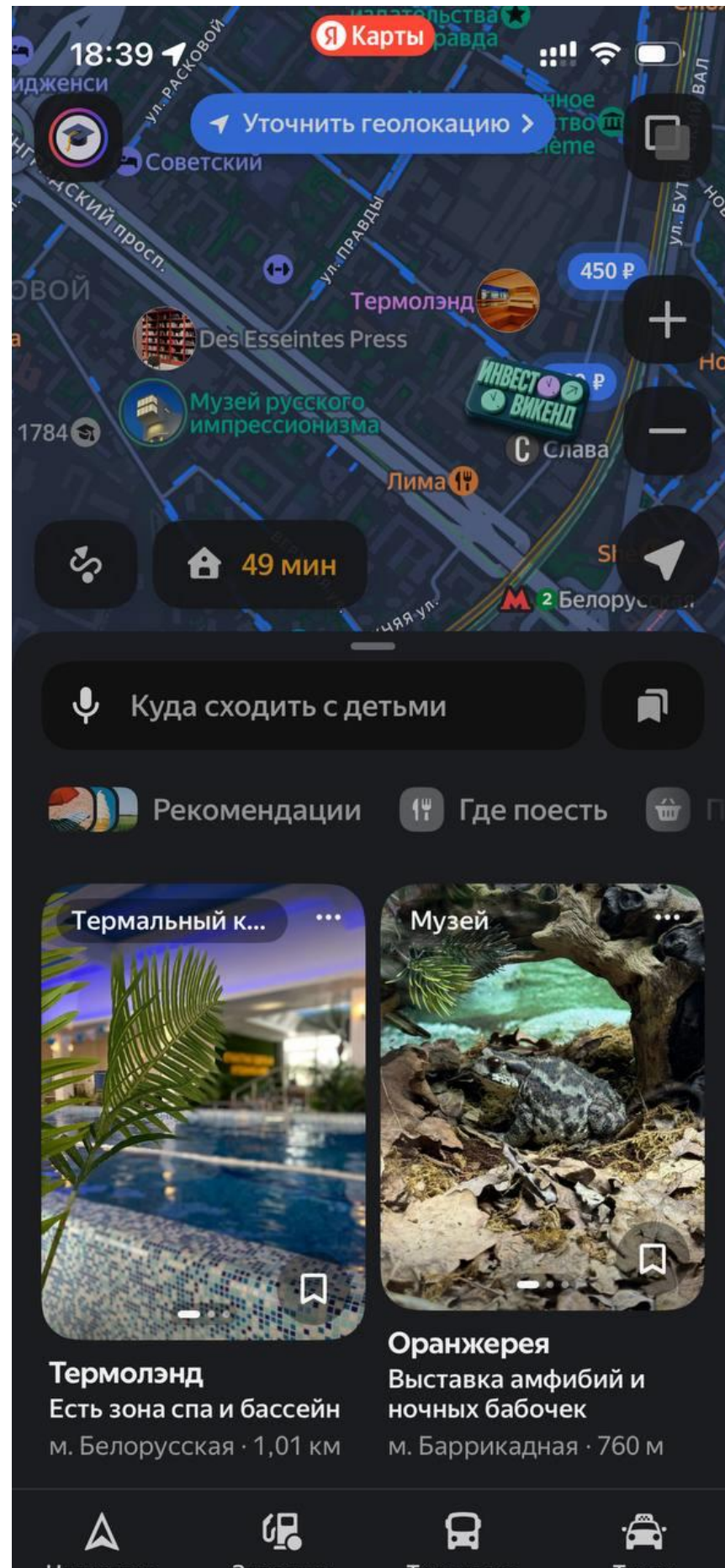


Есть много разных городов, и мы должны для каждой пары user – город насчитать оффлайн списки.

Мы не знаем заранее, в каком городе купит пользователь.

Для непопулярных городов еще меньше пользы

Проблема оффлайн списков с контекстом

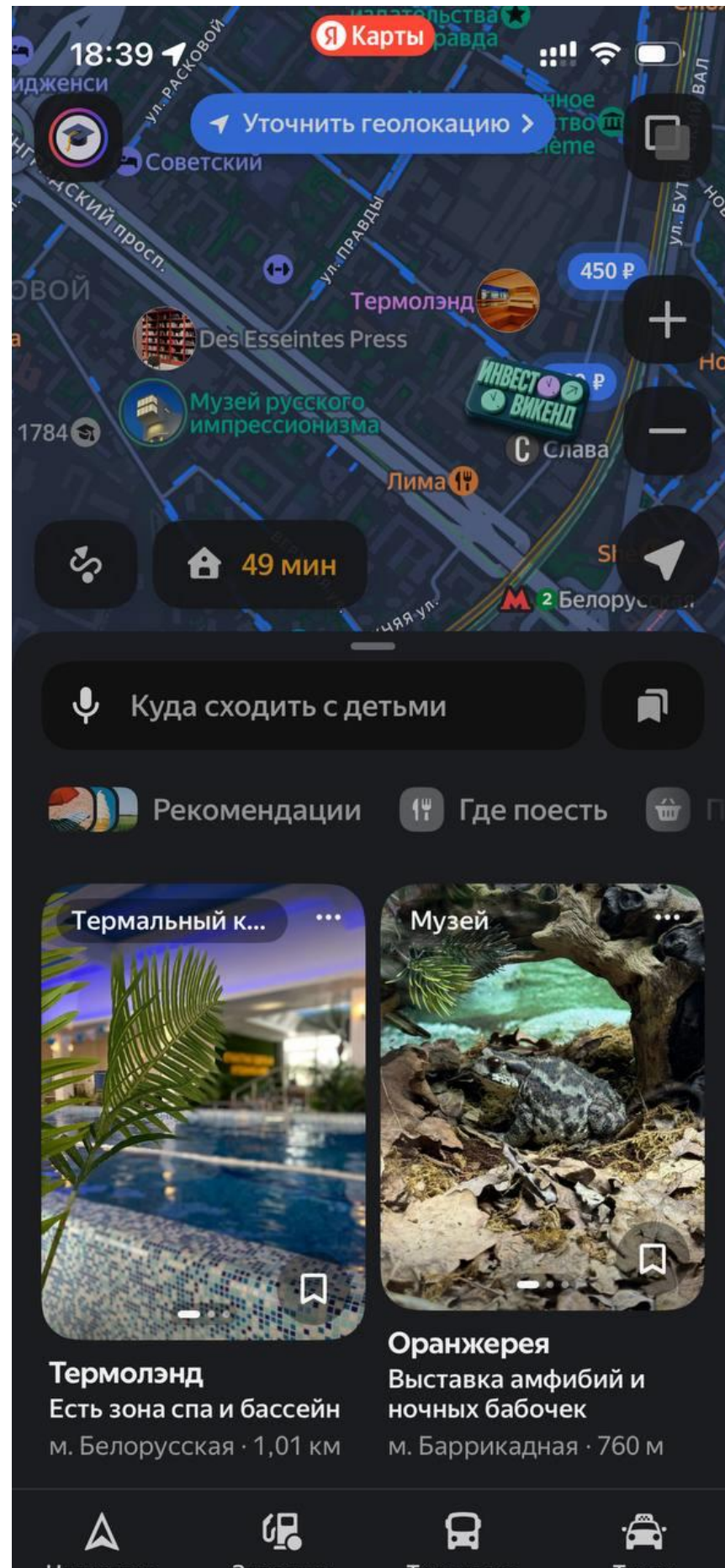


Есть много разных локаций (областей), и мы должны для каждой пары user – локация насчитать оффлайн списки.

Мы не знаем заранее, в какой локации будет пользователь.

Для непопулярных локаций еще меньше пользы

Проблема оффлайн списков с контекстом

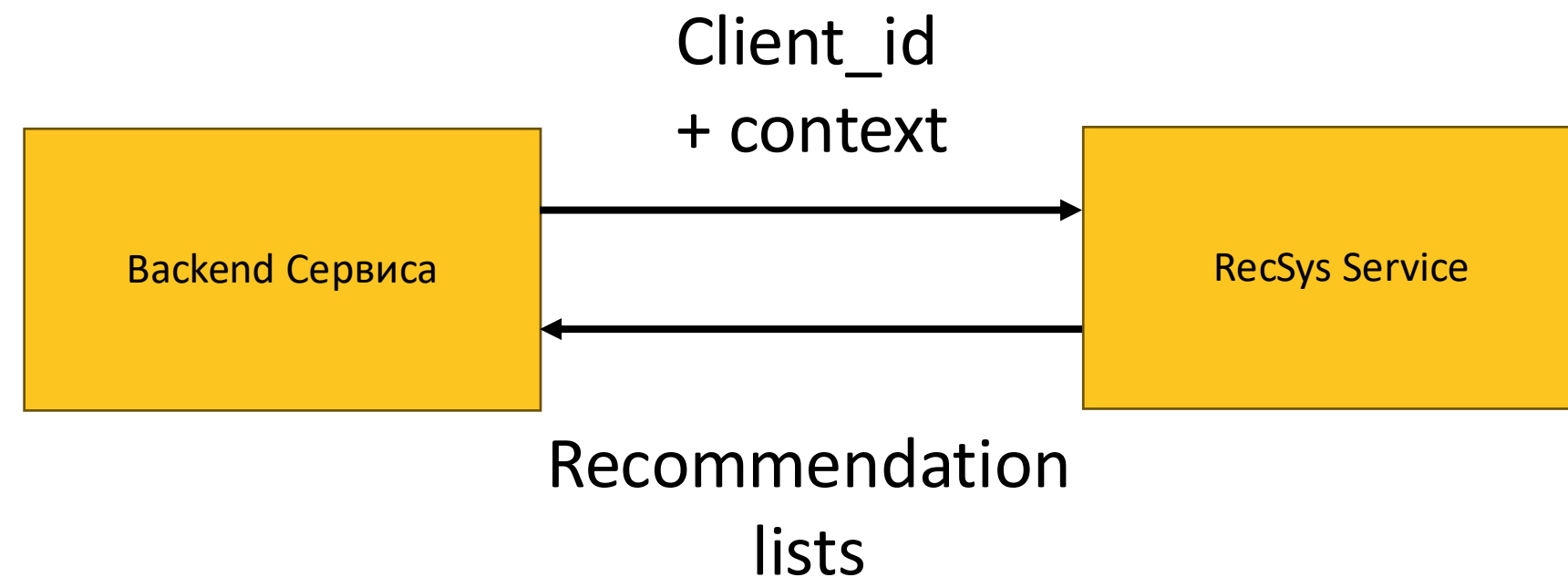
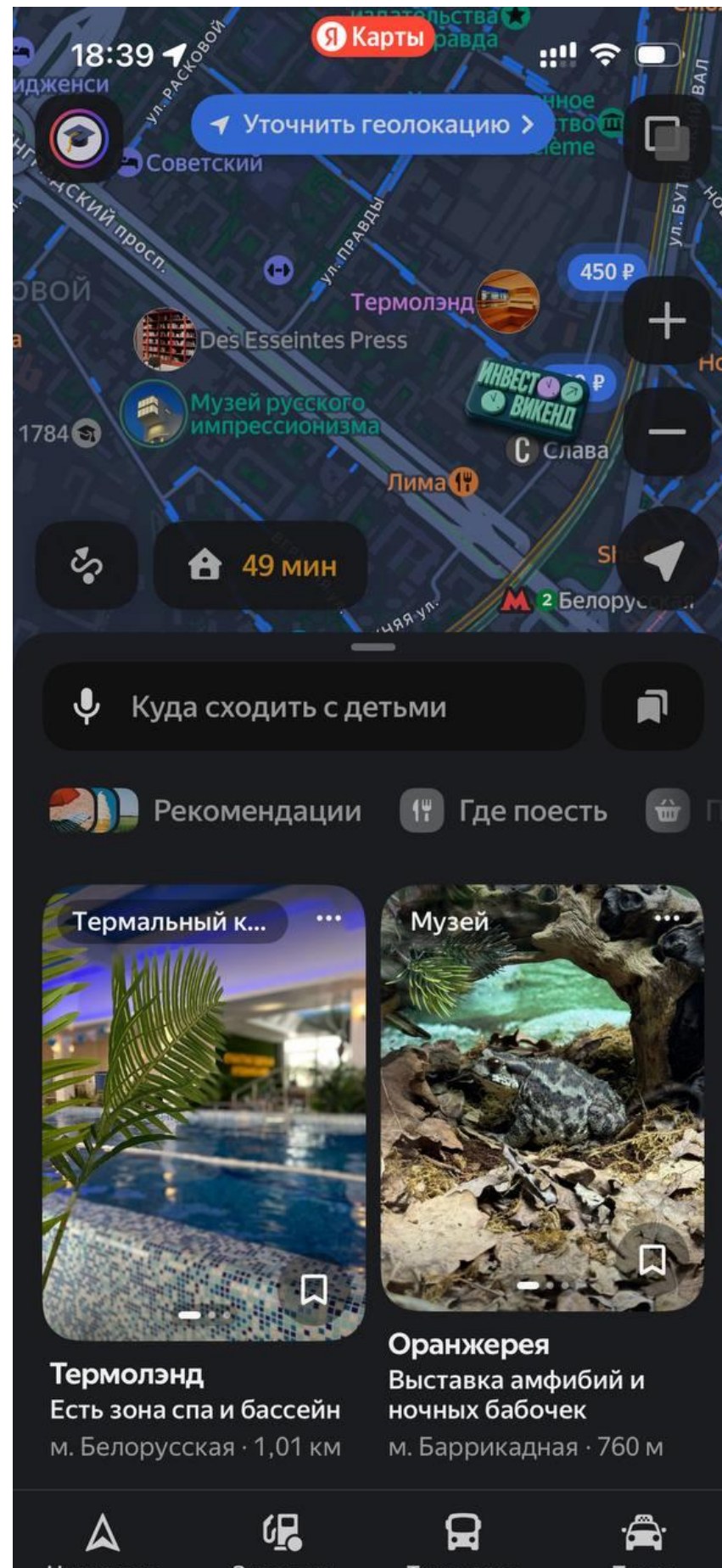


Можно решать эвристиками:

Насчитать списки только для ожидаемого «контекста» пользователя.

Например, пользователь из Петрозаводска, считаем список для него только в Петрозаводске, остальные - нет

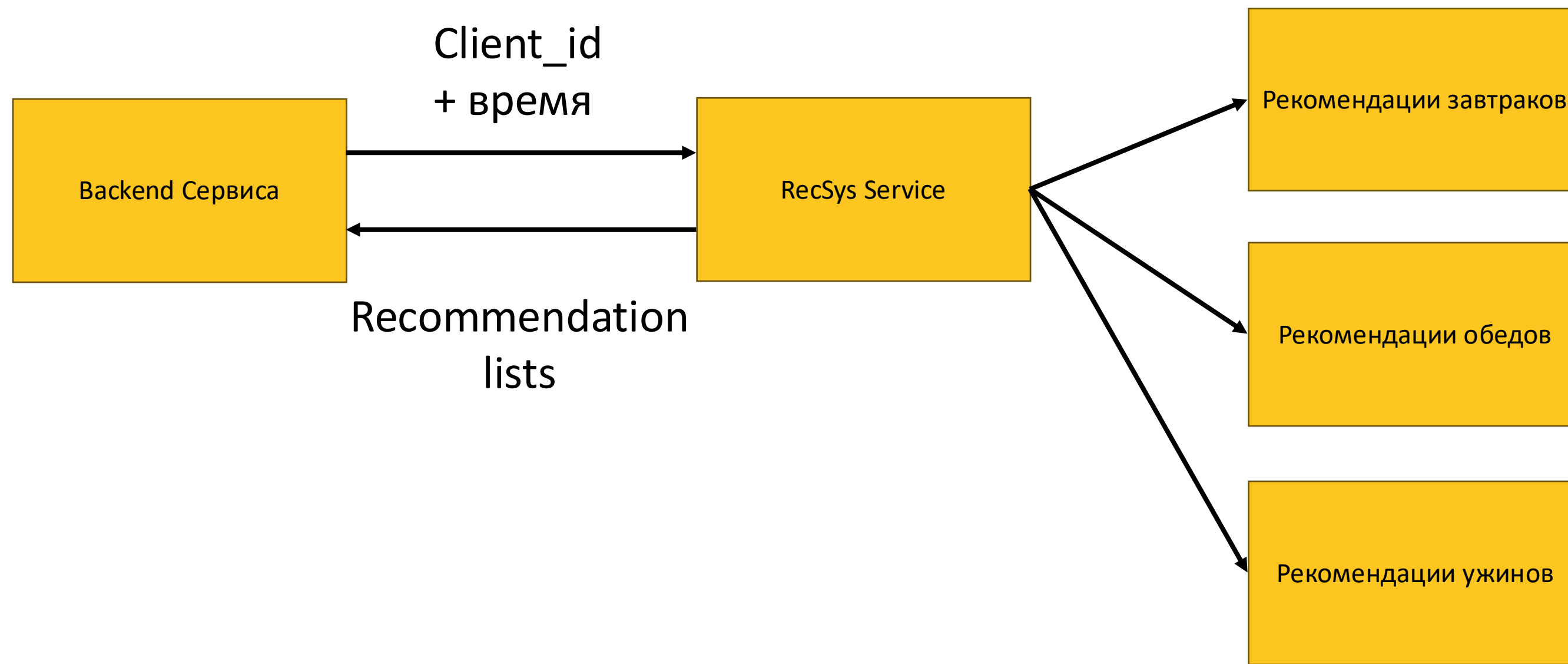
В таких ситуациях нужен онлайн инференс



Сценарии:

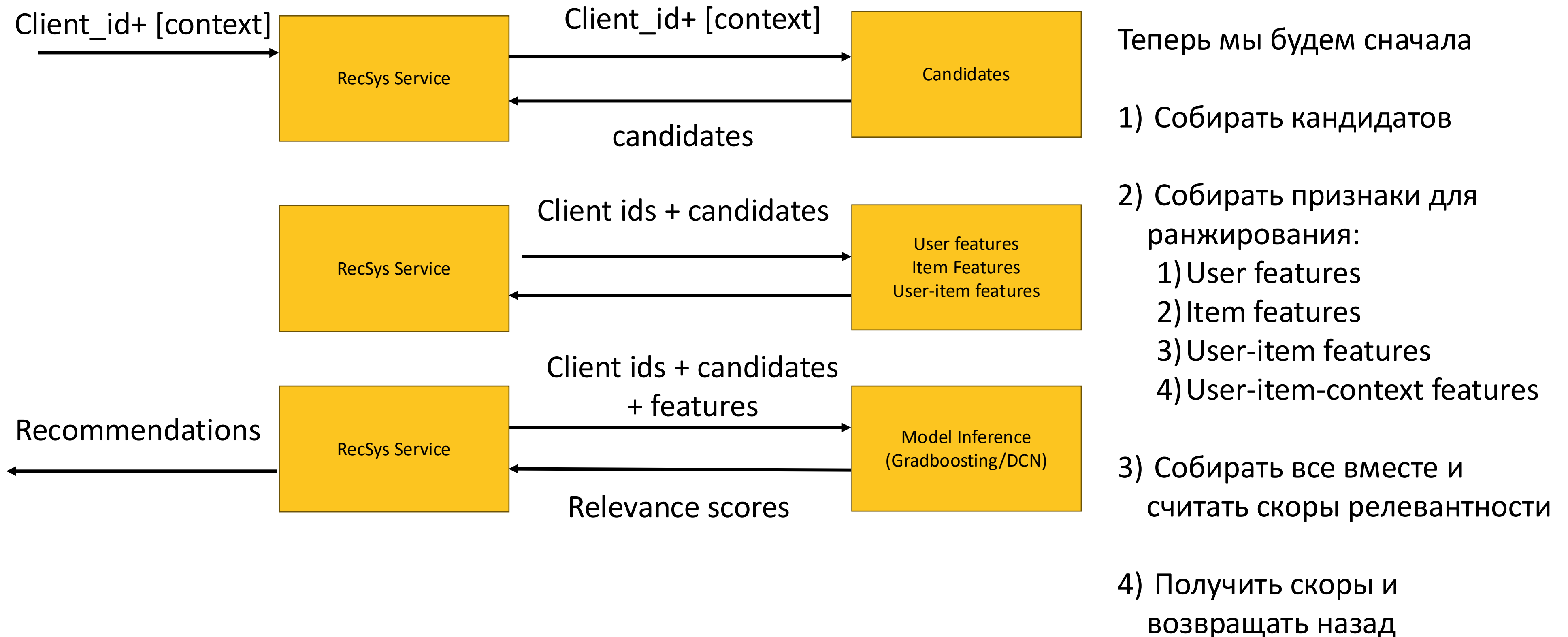
- 1) Получаем рекомендации для контекста (город/локация/магазин) – и в зависимости от выбранного показываем этот список (неперсонально)
- 2) Есть кандидаты для контекста, но мы их персонально переранжируем
- 3) Мы можем персонализировать кандидаты под контекст, и в зависимости от него генерировать выдачи. Предрассчитанных выдач не существует

В таких ситуациях нужен онлайн инференс



Простой пример персонализации в зависимости от времени суток, где мы выбираем разных кандидатов

Персональное ранжирование в онлайн



Персональное ранжирование в онлайн

Онлайн ранжирование:

Плюсы:

1. Можем подстраивать кандидатов под контекст
2. Считаем рекомендации, только если пользователь сошел
3. Появляется возможность радикально сократить опоздание по данным
4. Раскатка алгоритма происходит «мгновенно», не надо все пересчитывать
5. Можем учитывать контекст в смысле времени/девайса/текущего баланса и тд

Минусы:

1. Ограничены ожидаемым временем ответа, и сложностью модели
2. Намного более сложная реализация
3. Нет «фоллбека» на персональные рассчитанные оффлайн рекомендации
4. В некоторых кейсах может быть «оверкил»: если система не такая динамичная
5. Требуется больше компетенций у команды, чем просто ML-инженер (например, сервис на python может тормозить, нужны другие ЯП)

Тестирование онлайна легче

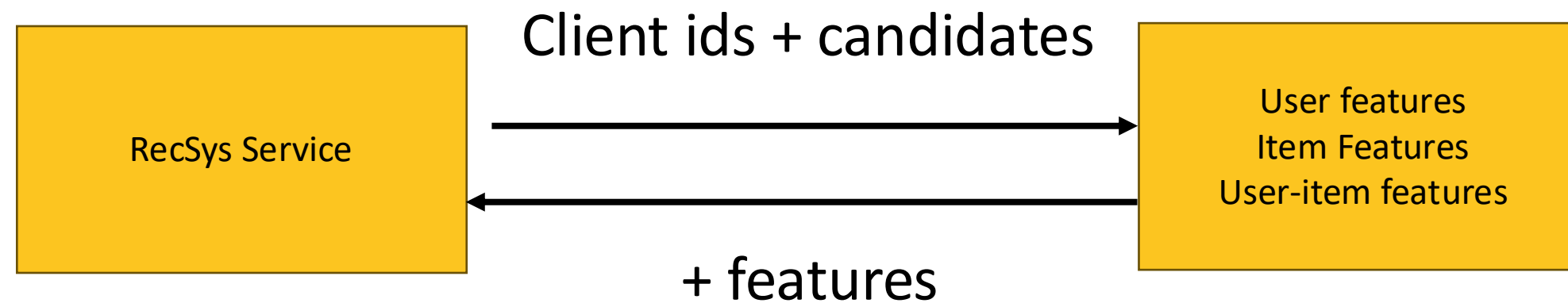
Отдельный плюс:

Если у нас есть 3 разных моделей, нам надо их протестировать в АБ тесте, тогда:

- 1) **В оффлайне:** надо настроить разбиение пользователей заранее, насчитать для разных юзеров разные алгоритмы
- 2) **В онлайнне:** мы настраиваем конфиг, и в зависимости от того, куда попал пользователь, выбираем, какой моделью ему проскорить – это намного более гибко

Если заранее неизвестно разбиение на тест (разбиение происходит в момент захода юзера), то онлайн намного более подходит, чем оффлайн.

Персональное ранжирование в онлайн



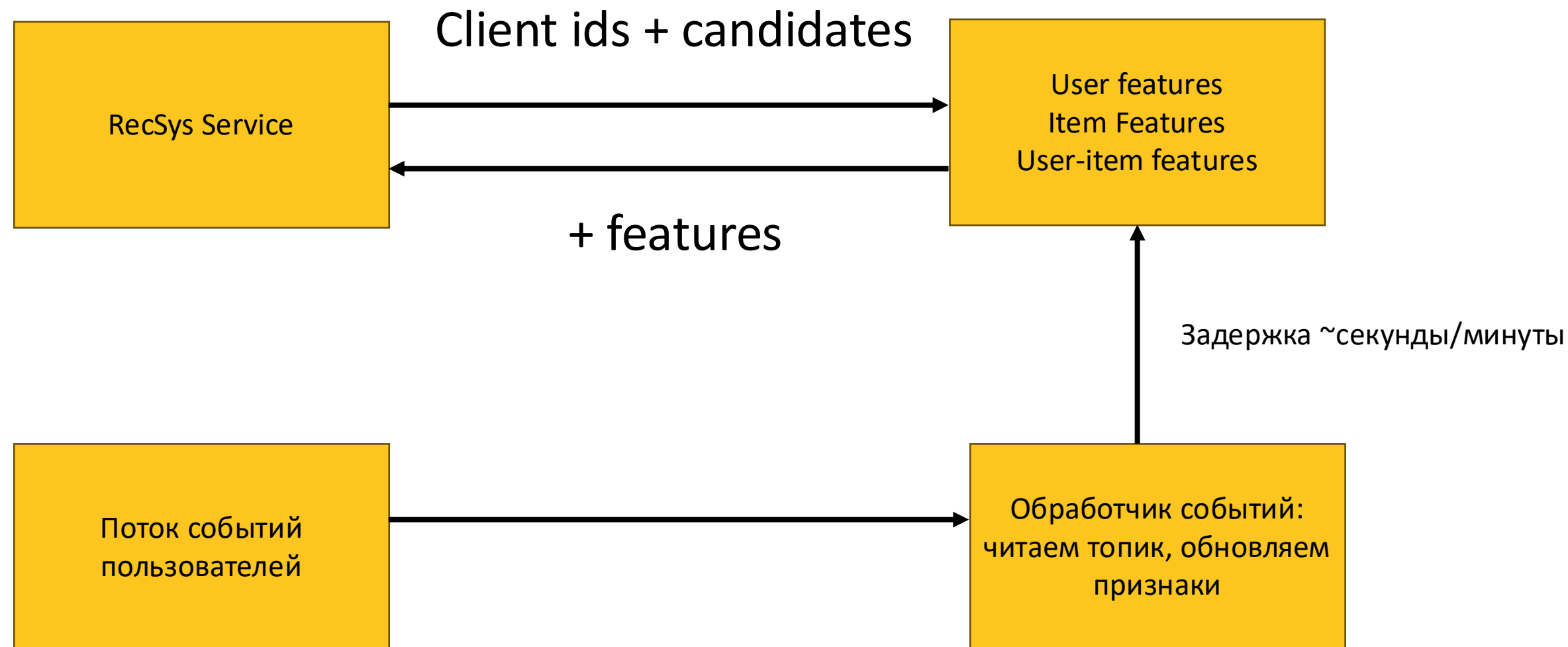
Есть два варианта:

- 1) User/item/user-item features обновляются раз в день, также в оффлайне по расписанию
- 2) Обновляем признаки с меньшей задержкой: это еще более хорошо, но еще более сложно

Даже первый вариант решает часть проблем.

В первом варианте также возможен recent_inference

Обновление признаков ближе к онлайну



- 1) Пользователь совершает клик
- 2) Клик попадает в поток событий
- 3) Обработчик событий вычитывает событие
- 4) Смотрит на текущее значение нужных признаков
- 5) Производит операцию их изменения

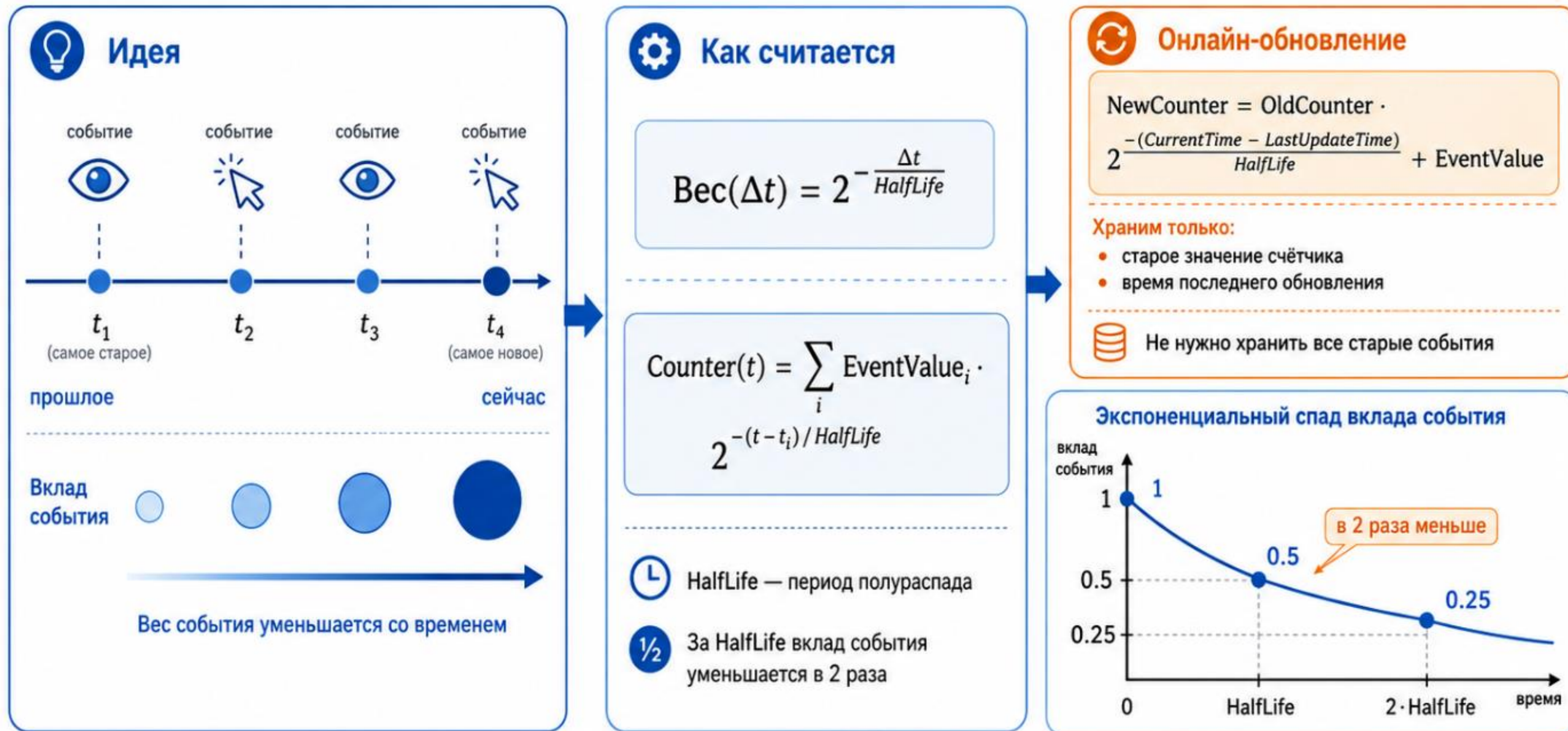
- 1) Пользователь совершает клик
- 2) проходит 20 секунд, ему нужны новые рекомендации
- 3) Рес сервис опять идет в фичи
- 4) В фичах уже учтен клик из п. 1
- 5) Рекомендации учитывают этот клик

Это все существенно сокращает «возраст рекомендаций» и «время с учета последнего события»

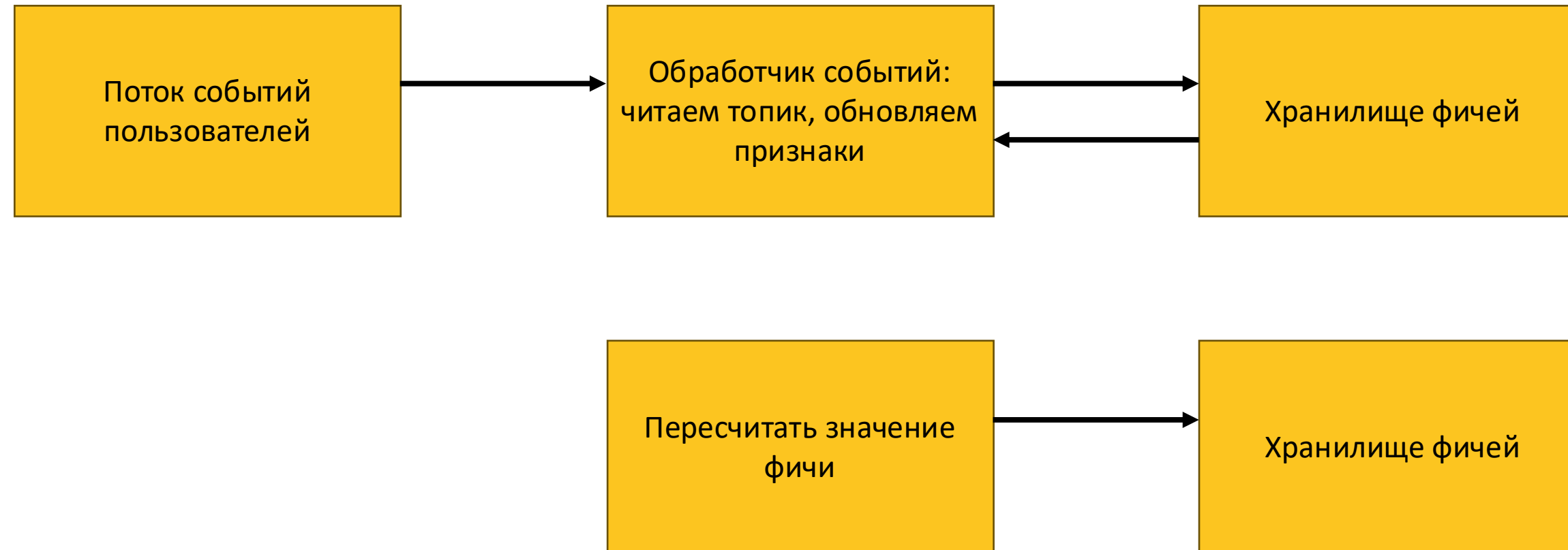
Идея экспоненциального счетчика для фичей

Счётчик с экспоненциальным затуханием

Старые события влияют меньше, новые — сильнее



Идея экспоненциального счетчика для фичей



Обработчик событий в случае экспоненциального счетчика:

- 1) Считывает текущее значение нужной фичи
- 2) Пересчитывает значение на новое значение
- 3) Сохраняет назад эту фичу

Профиль пользователя

Экспоненциальные счётчики же обновлять очень просто:

$$\text{NewCounterValue} = \text{OldCounterValue} * 2^{-(\text{CurrentTime} - \text{LastUpdateTime}) / \text{HalfLife}} + \text{EventValue}$$

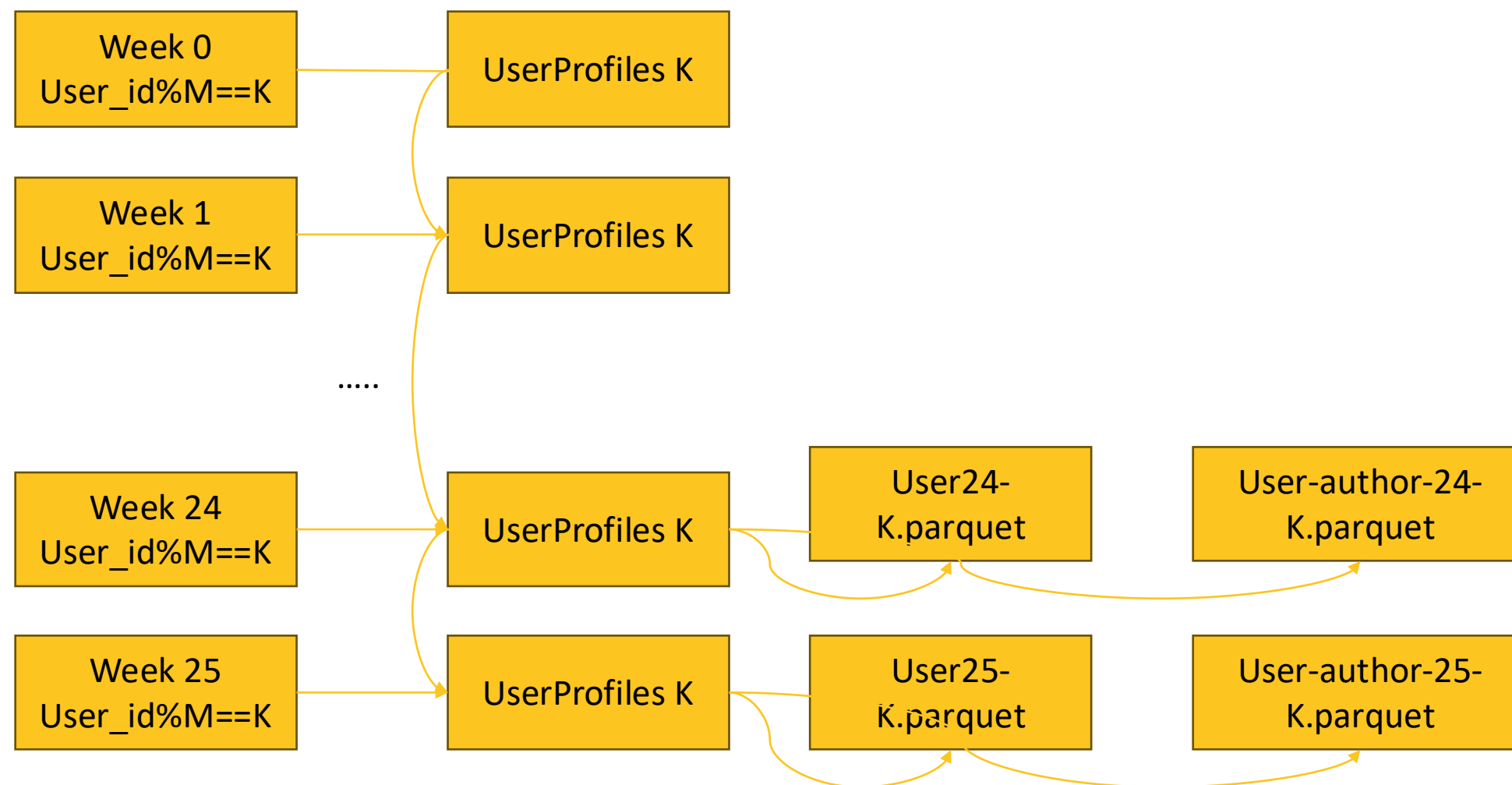
Создадим «профиль пользователя»:

- 1) Хранит в себе актуальные готовые значения признаков user-/user-item/author/category
- 2) Умеет обновлять состояние по одной строке данных взаимодействий
- 3) Умеет сохранять состояние в parquet

В нем:

- 1) Хранится словарь фичей по юзеру. Некоторые фичи обновляются экспоненциально, некоторые нет
- 2) В юзере есть словарь таких же фичей в разрезе items/автора/категорий, что делает фичи user-author/user-item..

Профиль пользователя

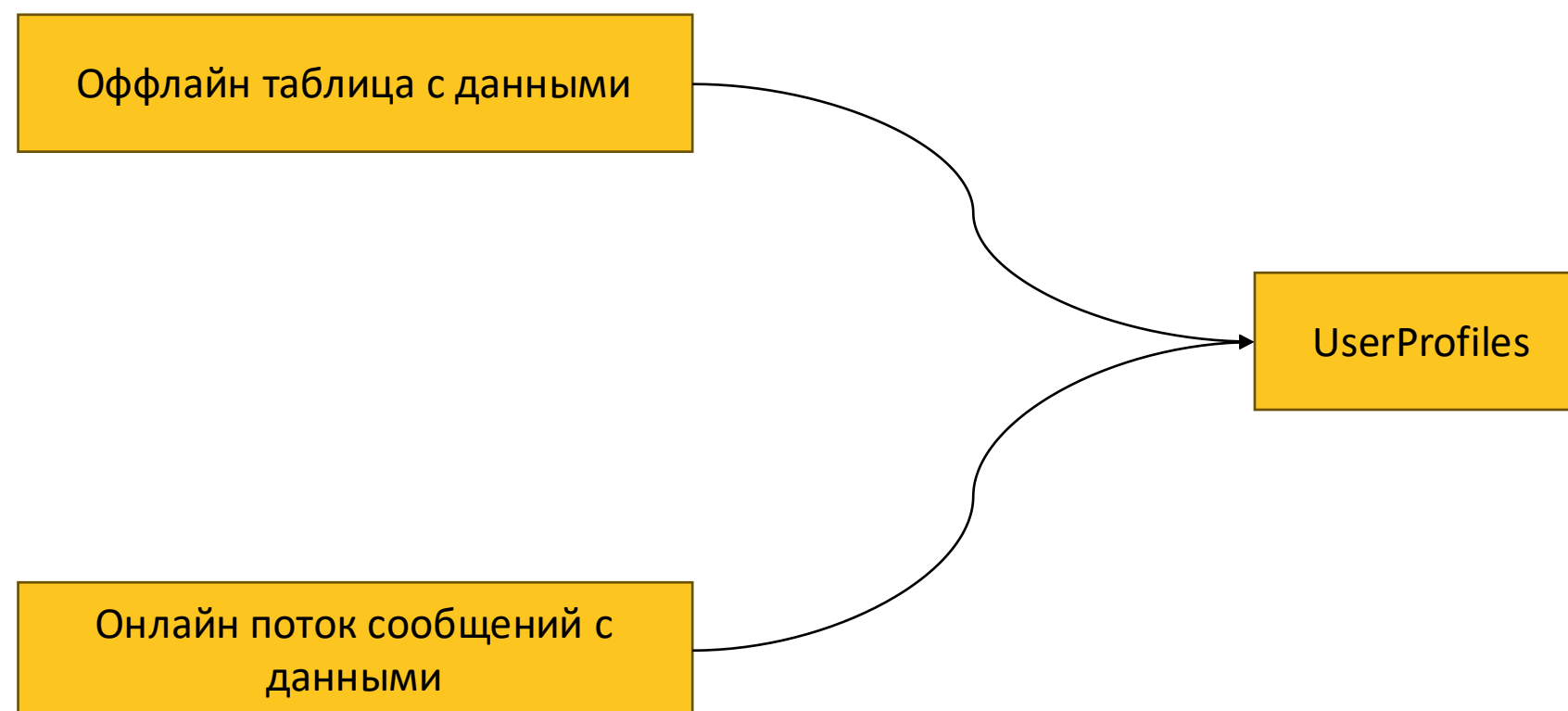


Мы берем некоторый M ($M=200/500/1000/5000$) и обрабатываем только юзеров с выбранным $k=0, \dots, m-1$

Чем меньше RAM, тем больше M , дольше время работы.

Фичи почти не меняются даже при $M=100000$

Онлайн/оффлайн Профиль пользователя



В идеале:

- 1) Есть некоторое событие по пользователю: `user-item-type-timestamp-[FEATURES]`
- 2) Оно хранится либо в оффлайн базе, либо случается в онлайн потоке
- 3) Событие учитывается для пересчета профиля пользователя
- 4) Если все события до Timestamp обработаны, то профиль содержит актуальные данные до Timestamp
- 5) Такой профиль можно использовать для сбора обучения и для инференса

Кэширование

Чтобы снизить число походок в БД (которое нагружает ее), можем сделать кэш:

- 1) Начать кэшировать (сохранять где-то в сервисе ответы из БД) рекомендации
- 2) Для каждого пользователя проверить наличие юзера в кэше. Если там есть юзер, взять рекомендации из кэша
- 3) Если юзера в кэше нет, сходить в БД и сохранить рекомендации в кэш

У кэша могут быть параметры в виде кол-ва юзеров максимального и времени хранения кэша. Как только время проходит, запись удаляется и надо будет заново идти в БД.

Плюсы:

- 1) снижаем нагрузку на БД, быстрее отвечаем пользователю

Минусы:

- 1) recent_inference и онлайн инференс менее эффективны (хотели бы обновить рекомендации, но сервис за ними не придет) – опять tradeoff

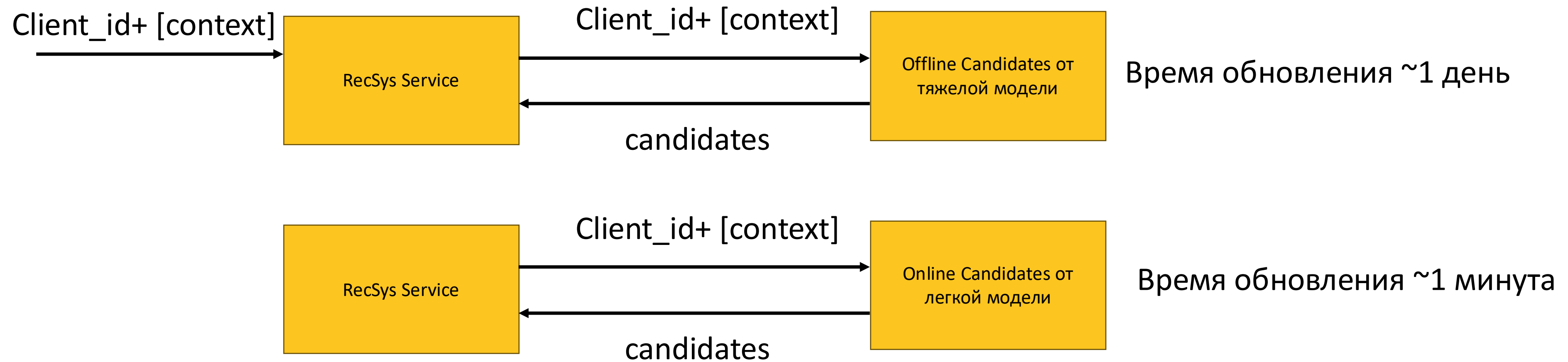
В такой системе есть много точек отказа, в идеале уметь работать с каждой точкой отказа.

Если совсем все сломалось, в идеале иметь дефолтный список рекомендаций, которые где-то лежат в продукте/в самом начале рек. Сервиса.

Фоллбек:

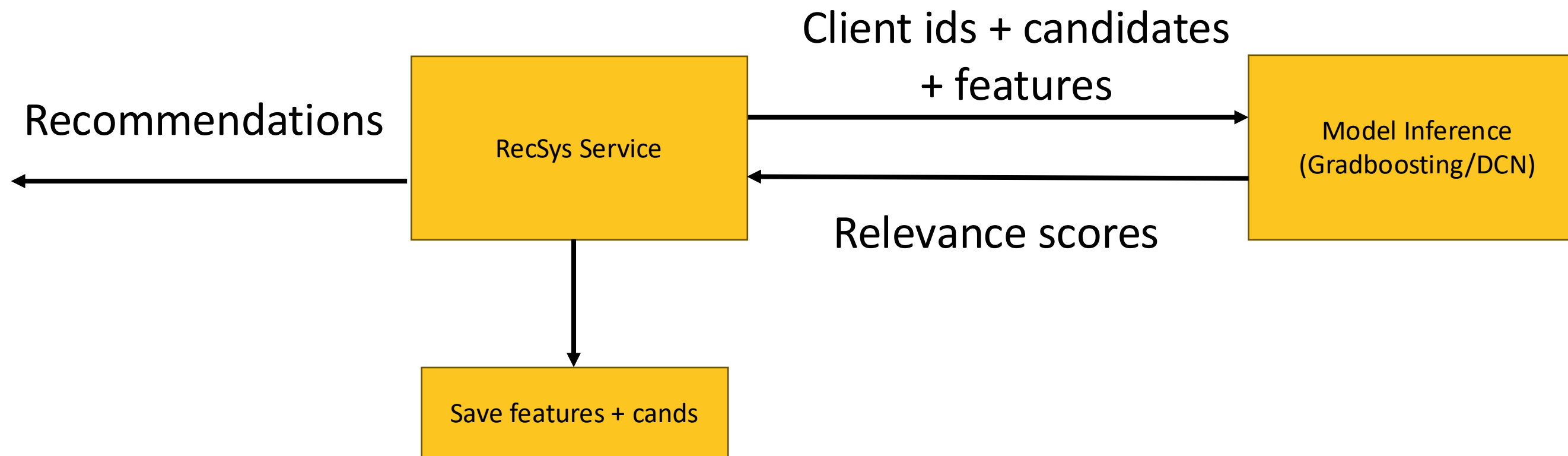
- 1) Most popular – самый простой и дефолтный
- 2) Socdem – если все-таки пол/возраст известны, можем выдавать хотя бы списки по ним
- 3) Только кандген – если есть кандидаты, но нет ранжирования -> показываем кандидатов as is

Разная степень онлайнности кандидатов



Дальше все это смешивается

Концепция log&wait



Здесь мы проделали большую работу:

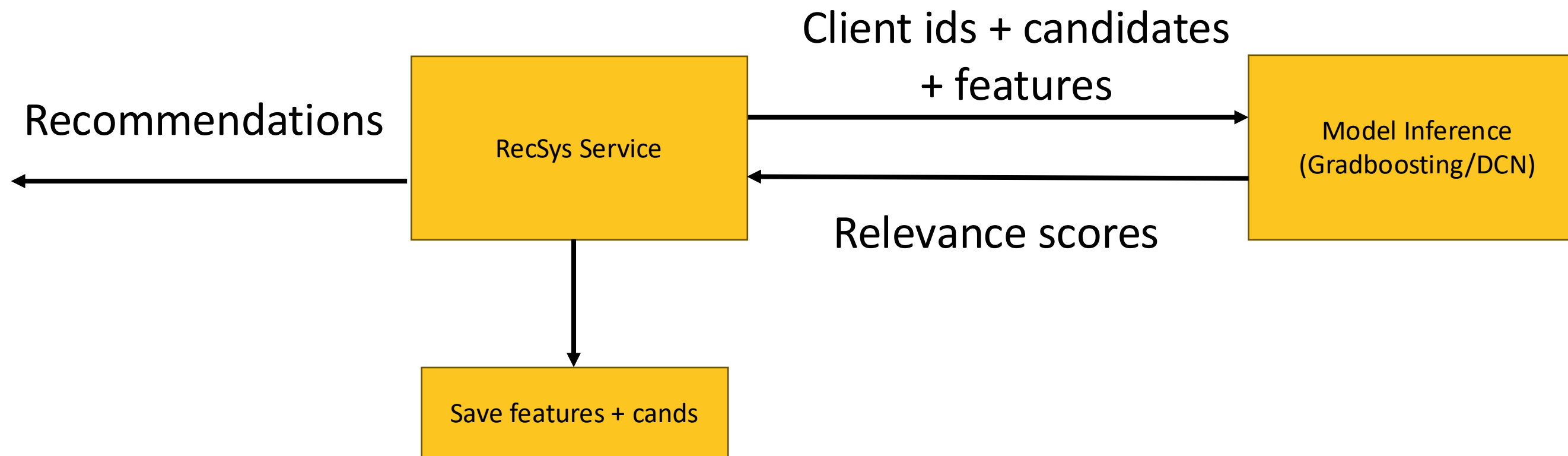
- 1) Собрали актуальных кандидатов
- 2) Собрали все признаки, в том числе самые актуальные

Мы можем залогировать набор user-item-timestamp-[FEATURES]

Потом на этот набор можем “приклеить” target =>

Получим датасет для обучения ранкера

Концепция log&wait



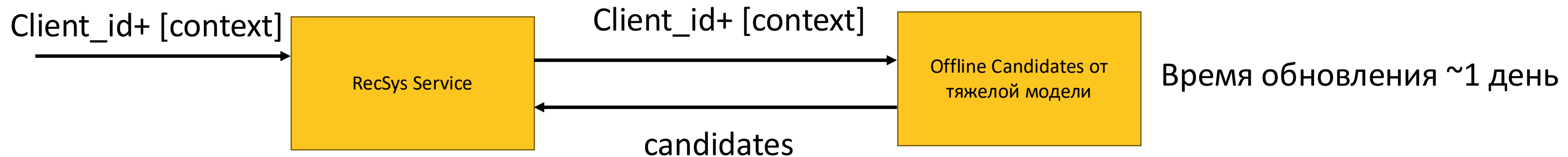
Плюсы log&wait:

- 1) У нас накапливается готовый датасет для обучения с актуальными фичами
- 2) Мы можем реализовать фичу и начать ее логировать, но не использовать
- 3) Датасет для инференса/обучения довольно сильно совпадают

Минусы:

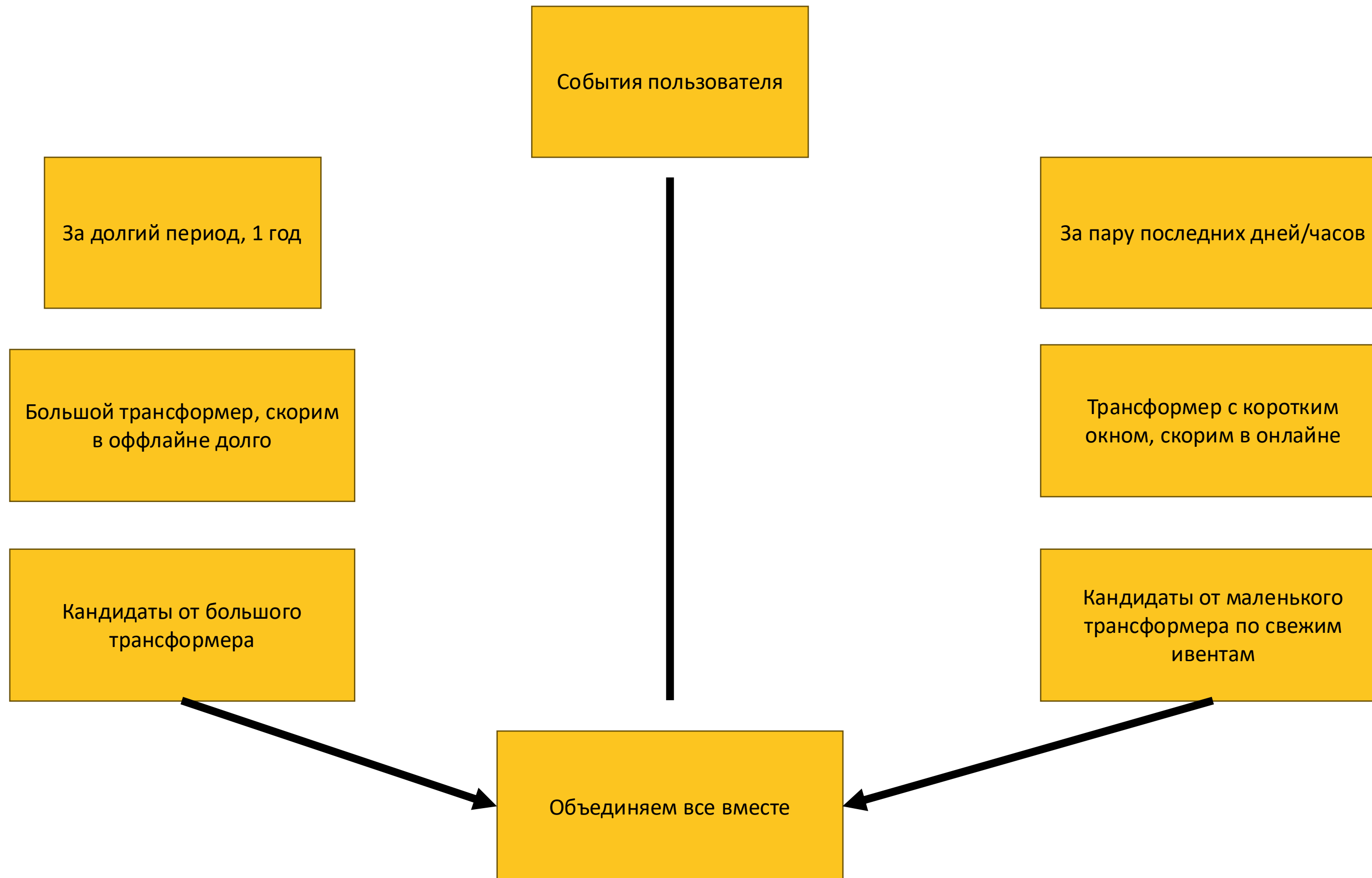
- 1) Мы должны ждать накопления логов по новым фичам
- 2) Мы можем остаться без альтернативного оффлайн способа сбора датасета
- 3) Это может быть непросто реализовать
- 4) Нужно быть аккуратными с версиями фичей

Нюансы с трансформером: условный инференс



- 1) Мы можем получить user vector из SASRec-like модели
 - 2) Все items мы можем разделить на контексты. Например, матрицу айтеров из нужной категории, из нужного города, и т. Д.
 - 3) Мы можем считать $\text{score}(u, i) = \text{user_vector} * \text{item_vector}$ только среди нужных айтеров (нужного города, категории)
 - 4) На выходе будем иметь рекомендации от трансформера только тех айтеров, которые нужны.
- Кейс: нужны рекомендации только из магазина X. Берем айтеры из магазина X, и скорим только из через SASRec-like

Нюансы с трансформером: легкий и тяжелый

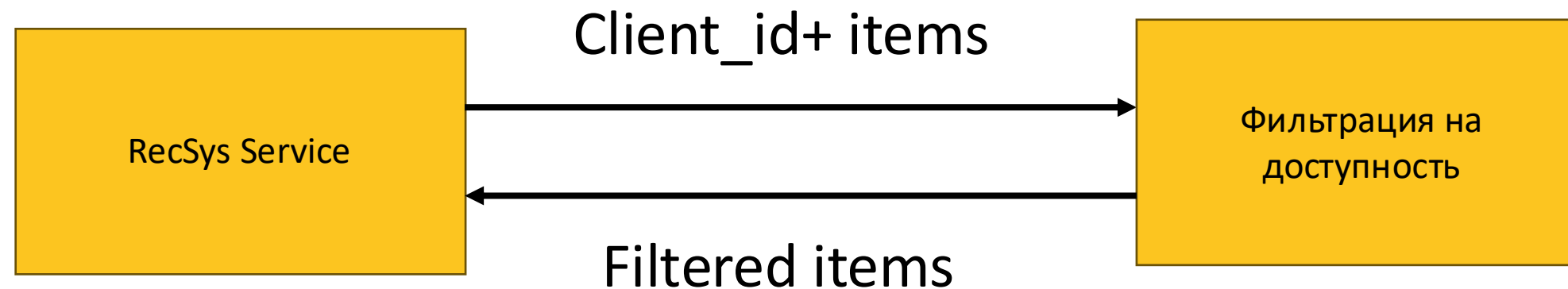


Пост-процессинг

RecSys Service

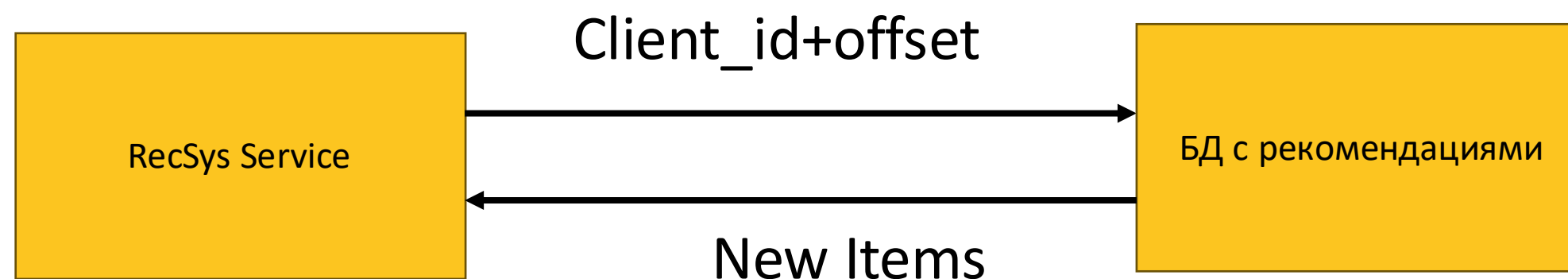
- 1) Отфильтровать выдачу на доступные айтемы
- 2) Сделать диверсификацию
- 3) Забустить рекламу
- 4) Передать скоры в еще один сервис (например, для определения персональной цены)
- 5) Прогнать айтемы на некоторый фильтр

Пост-процессинг по фильтрации на доступность

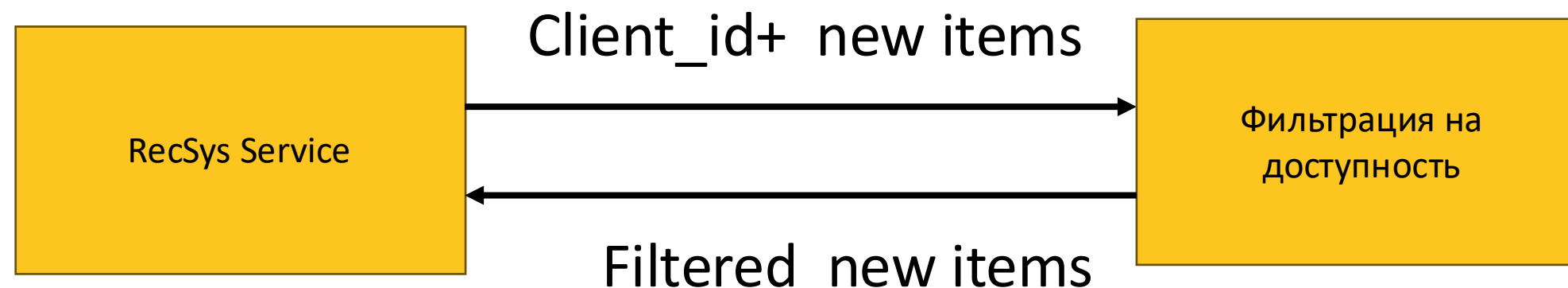


В случае сильной фильтрации мы можем дозапрашивать/досчитывать айтемы.

Можем контролировать метрику «доля фильтрации»



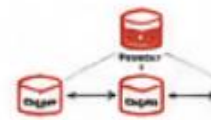
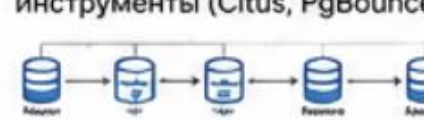
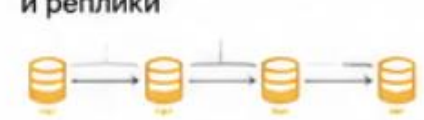
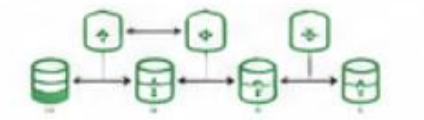
Как можно уменьшать фильтрацию?



1) Оставлять в моделях айтемы, которые реже фильтруются по статистике

2) Оставлять в моделях популярные айтемы (корреляция с доступностью)

Сравнительная таблица СУБД для хранения фичей и рекомендаций

СУБД	 Redis	 PostgreSQL	 Cassandra	 ClickHouse	 MongoDB	 ScyllaDB
Тип базы данных	Key-Value хранилище в памяти	Реляционная (SQL)	Ширококолонночная (NoSQL)	Колонночная (OLAP)	Документная (NoSQL)	Ширококолонночная (NoSQL)
Как данные хранятся	В памяти (поддерживает персистентность на диск). Структуры: строки, хэши, списки, множества, отсортированные множества, биты, гиперлоглоги и др.	Таблицы (строки и столбцы) с жесткой схемой. Поддержка индексов, связей, транзакций.	Данные хранятся в виде таблиц с динамической схемой (строки и столбцы). Оптимизирована для больших объемов данных и распределенного хранения.	Данные хранятся по столбцам. Оптимизирована для аналитических запросов и агрегаций.	Документы (BSON) в коллекциях. Гибкая схема, индексирование любых полей.	Строки в таблицах с фиксированными колонками. Совместима с Cassandra Query Language (CQL).
Подходит для хранения	 Кэш фичей (онлайн-доступ)  Сессии, счетчики, рекомендации в реальном времени	 Профили пользователей и фичи (онлайн/офлайн)  Рекомендации, транзакционные данные	 Фичи в офлайне и онлайн (большие объемы)  Рекомендации, история событий	 Агрегированные фичи  Аналитика рекомендаций, отчеты, офлайн-анализ	 Профили, фичи (гибкая структура)  Контентные данные, рекомендации	 Фичи и события (большие объемы)  Рекомендации, история взаимодействий
Плюсы	✓ Очень высокая скорость доступа (in-memory) ✓ Простота использования ✓ Богатые структуры данных ✓ Поддержка Pub/Sub, Lua-скриптов	✓ Надежность и ACID транзакции ✓ Сложные запросы (JOIN, агрегации и т.д.) ✓ Гибкость и расширяемость ✓ Большое сообщество	✓ Высокая отказоустойчивость ✓ Горизонтальная масштабируемость ✓ Высокая скорость записи и чтения ✓ Хорошо работает на больших объемах данных	✓ Очень высокая скорость аналитических запросов ✓ Сжатие данных ✓ Поддержка сложных агрегаций ✓ Оптимизирована для колонночных запросов	✓ Гибкая схема ✓ Легко масштабируется ✓ Удобная работа с вложенными данными ✓ Быстрая разработка	✓ Очень высокая производительность ✓ Низкие задержки ✓ Линейная масштабируемость ✓ Совместимость с Cassandra (тот же язык запросов)
Минусы	✗ Ограниченный объем памяти (дорого для больших объемов) ✗ Персистентность не так надежна, как в дисковых БД ✗ Нет сложных запросов	✗ Масштабирование по горизонтали сложнее (чем NoSQL) ✗ Может быть медленнее для очень больших объемов и высоких нагрузок	✗ Сложная модель данных для новичков ✗ Ограничения в сложных запросах и JOIN ✗ Консистентность настраивается (BASE)	✗ Не подходит для частых обновлений и записей ✗ Нет поддержки транзакций (как в OLTP БД) ✗ Не для онлайн-сервисов с низкой задержкой записи	✗ Менее строгая консистентность (по умолчанию) ✗ Агрегации могут быть тяжелыми при больших объемах	✗ Сложность настройки и эксплуатации ✗ Меньше инструментов и расширений, чем у Cassandra ✗ Молодая экосистема
Опенсорс	Да (BSD-3)	Да (PostgreSQL License)	Да (Apache 2.0)	Да (Apache 2.0)	Да (SSPL)	Да (Apache 2.0)
Масштабирование	Репликация (master-slave) и кластеризация (Redis Cluster) 	Репликация (master-slave), шарднинг через внешние инструменты (Citrus, PgBouncer) 	Линейное горизонтальное масштабирование (добавление узлов) 	Масштабирование через шардирование и реплики 	Репликация (Replica Set), шарднинг (Sharded Cluster) 	Линейное горизонтальное масштабирование (добавление узлов) 
Тип нагрузки	 Высокая нагрузка на чтение/запись (низкие задержки)	 Смешанная нагрузка (чтение/запись)	 Высокая нагрузка на запись и чтение больших объемов	 Аналитические запросы, большие сканирующие выборки	 Смешанная нагрузка, гибкие запросы	 Очень высокая нагрузка, низкие задержки
Когда использовать для фичей и рекомендаций	Онлайн-сервисы, кэширование фичей, быстрый доступ к рекомендациям в реальном времени	Хранение профилей, связанных данных, фичей с сложными связями и логикой	Хранение больших объемов фичей и событий, история взаимодействий, рекомендации в офлайне и онлайн	Анализ фичей, офлайн-рекомендации, построение отчетов и дашбордов	Хранение фичей с гибкой структурой, контентные рекомендации	Хранение больших объемов фичей и событий с высокими требованиями к скорости и масштабируемости

💡 Выбор зависит от сценария: для онлайн-сервисов с низкими задержками — Redis; для сложных запросов и транзакций — PostgreSQL; для больших распределенных систем — Cassandra/ScyllaDB; для аналитики и офлайн-обработки — ClickHouse; для гибкости и быстрой разработки — MongoDB.

Спасибо за
внимание